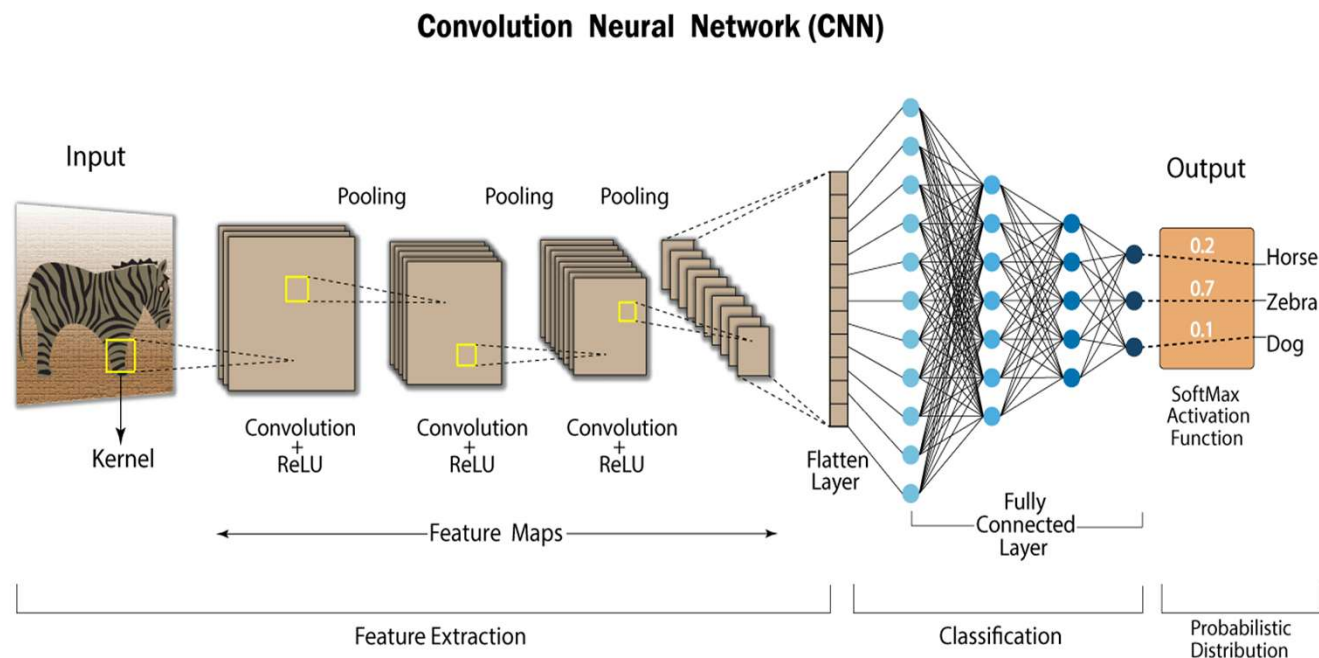

VISUAL OBJECT CLASSIFICATION DETECTION AND SEGMENTATION

IMAGE CLASSIFICATION

Image classification is the task of assigning a single label to an input image $\mathbf{X} \in R^{H \times W \times C}$, where H, W, C represent height, width, and color channels respectively.



<https://www.analyticsvidhya.com/blog/2022/03/basics-of-cnn-in-deep-learning/>

THE CONVOLUTIONAL OPERATION:

The core of modern classification is the Convolutional Neural Network (CNN). Mathematically, a 2D convolution in a deep learning context (technically a cross-correlation) for an input $\mathbf{X}$ and a filter (kernel) $\mathbf{K} \in R^{k \times k \times C}$ is defined as:

$$(X * K)_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \sum_{c=0}^{C-1} X_{i+m,j+n,c} \cdot K_{m,n,c}$$

THE SOFTMAX LAYER AND OBJECTIVE FUNCTION

To transform the final feature map into class probabilities, we apply a Softmax function to the logit vector $\mathbf{z}$:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

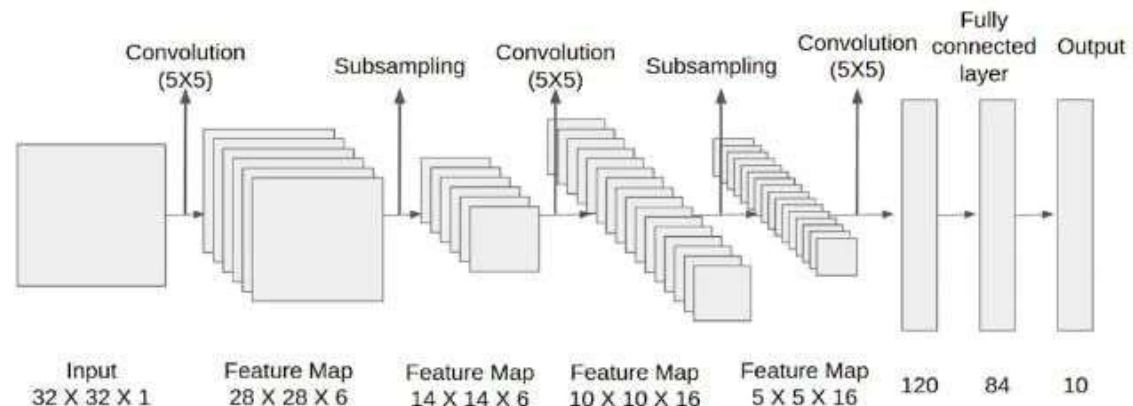
The model is optimized using the Categorical Cross-Entropy (CCE) loss, which measures the divergence between the true distribution $\mathbf{y}$ and predicted distribution $\hat{\mathbf{y}}$:

$$\mathcal{L}_{\text{CCE}} = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

Note: In a classification context, y_i is typically a one-hot encoded vector where only the ground-truth index is 1, simplifying the loss to $\mathcal{L} = -\log(\hat{y}_{\text{target}})$.

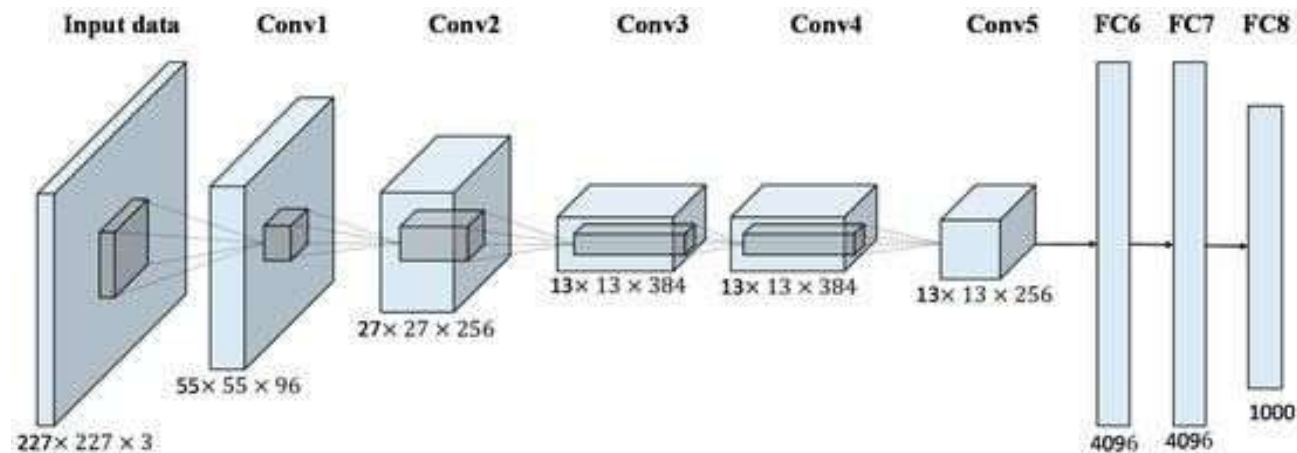
CLASSIFICATION ARCHITECTURES EVOLUTION - LENET-5

LeNet-5, introduced by Yann LeCun et al. in 1998, was one of the pioneering CNN architectures. It was designed for handwritten digit recognition and consisted of two convolutional layers, followed by average pooling, fully connected layers, and a softmax output layer. LeNet-5's lightweight architecture and efficient use of shared weights laid the foundation for modern CNNs, influencing subsequent developments in the field.



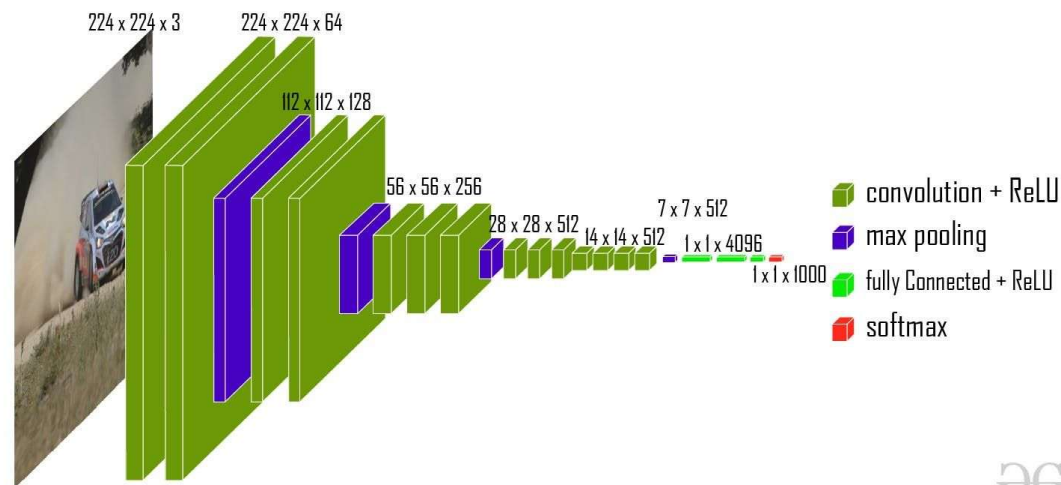
CLASSIFICATION ARCHITECTURES EVOLUTION - ALEXNET

AlexNet developed by Alex Krizhevsky et al. in 2012, made significant strides in image classification. It won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012 and achieved a significant improvement over previous methods. AlexNet featured a deeper architecture with multiple convolutional layers, introduced the concept of ReLU activations, and employed techniques like dropout for regularization. Its success paved the way for the resurgence of deep learning in computer vision.



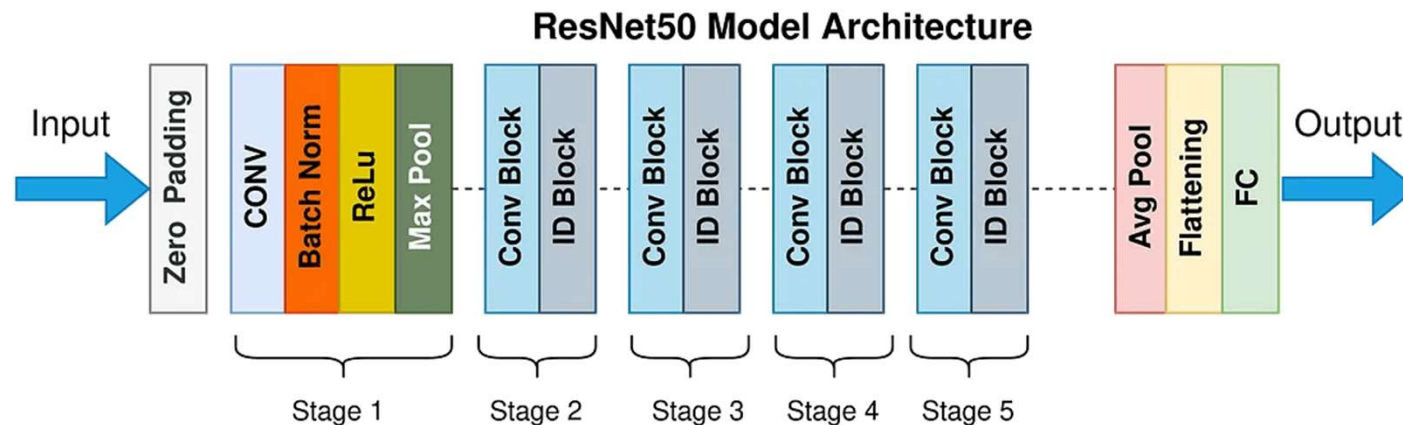
CLASSIFICATION ARCHITECTURES EVOLUTION - VGGNET

VGGNet, proposed by Karen Simonyan and Andrew Zisserman in 2014, introduced deeper architectures with up to 19 layers. VGGNet's key characteristic was the consistent use of 3x3 convolutional filters throughout the network, which enabled better modeling of complex visual patterns. The VGGNet architecture is known for its simplicity and effectiveness, and it has served as a baseline for subsequent CNN models.

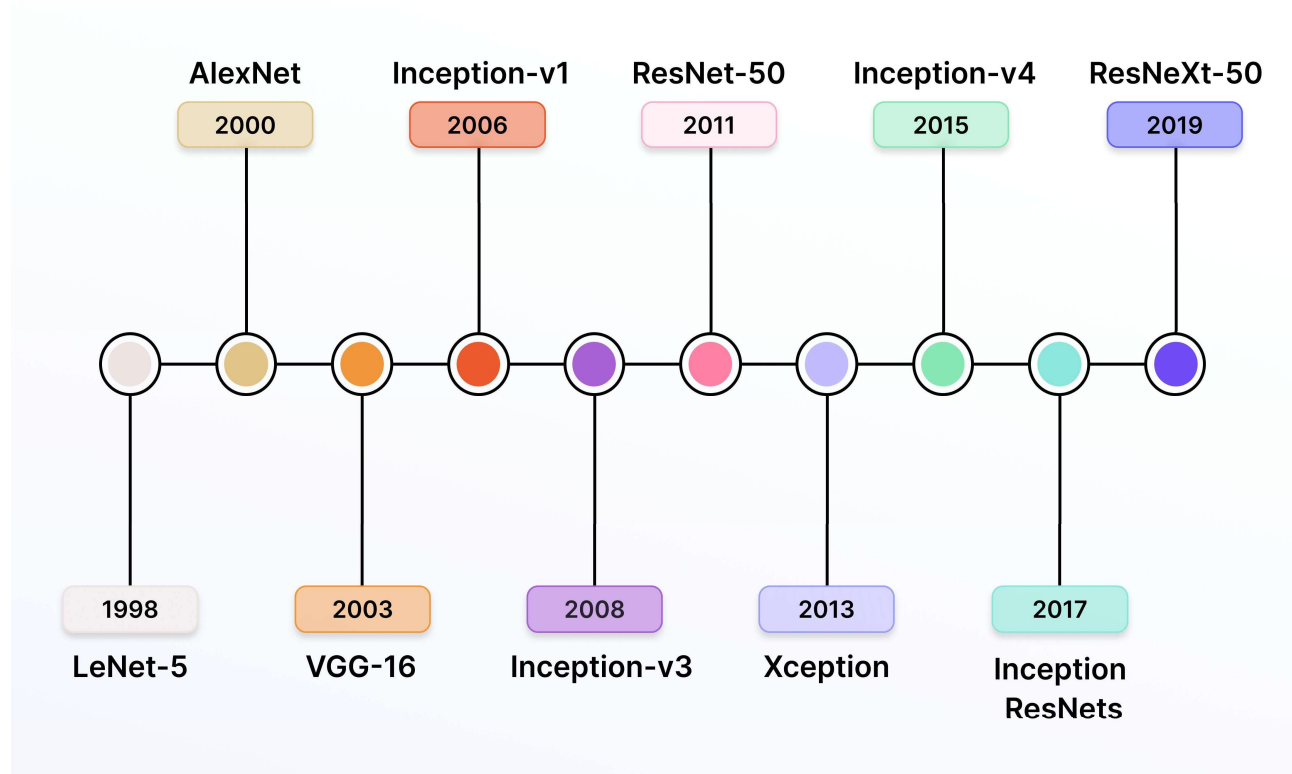


CLASSIFICATION ARCHITECTURES EVOLUTION - RESNET

ResNet developed by Kaiming He et al. in 2015, brought significant advancements in training very deep neural networks. ResNet introduced the concept of residual connections, allowing information to bypass certain layers and alleviating the vanishing gradient problem. This breakthrough enabled the successful training of CNNs with 50, 100, or even more layers. ResNet's skip connections revolutionized deep learning architectures and enabled the training of extremely deep and accurate networks.



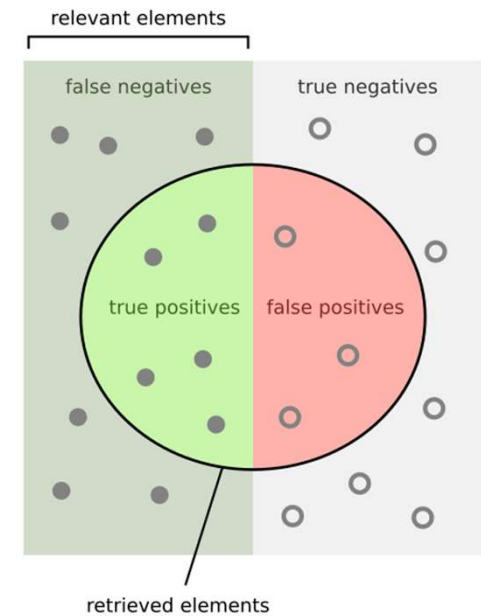
CLASSIFICATION ARCHITECTURES EVOLUTION



CLASSIFICATION METRICS - PRECISION / RECALL

$$\text{Precision} = \frac{\text{Relevant retrieved instances}}{\text{All **retrieved** instances}}$$

$$\text{Recall} = \frac{\text{Relevant retrieved instances}}{\text{All **relevant** instances}}$$



How many retrieved items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are retrieved?

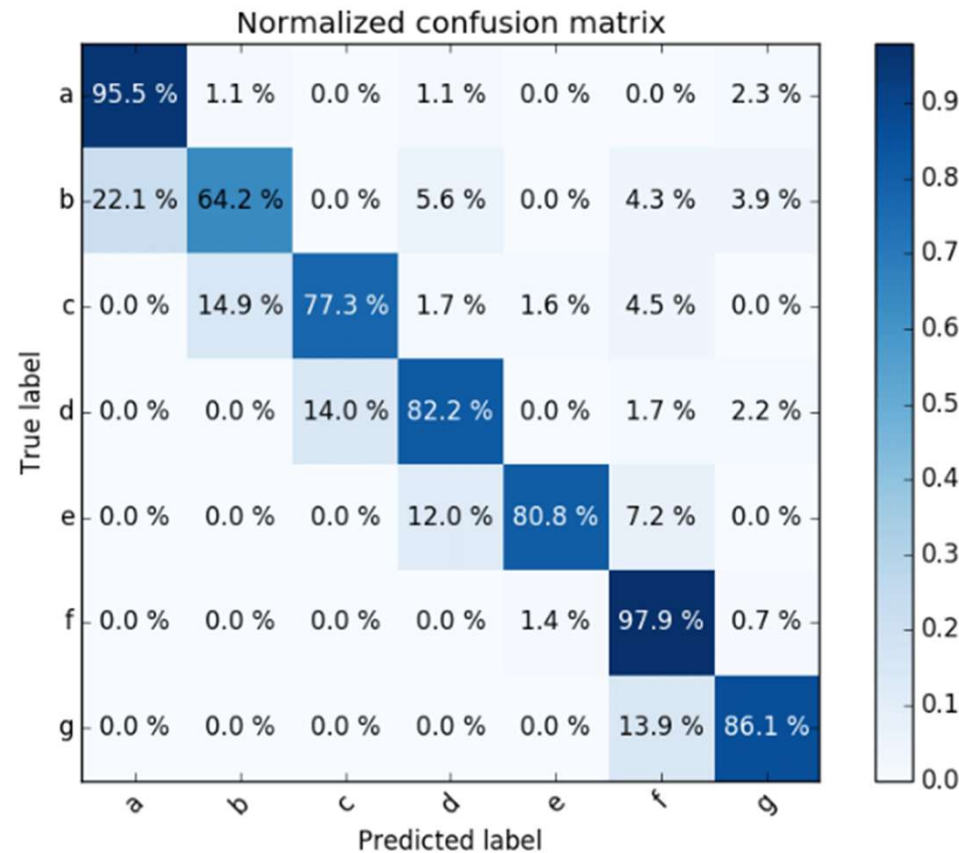
$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

CLASSIFICATION METRICS - F1 SCORE

- For most application it is highly desired to have a single Figure of merit for quantifying the performance of a classifier
- The F1 score is the harmonic mean of the precision and recall. It thus symmetrically represents both precision and recall in one metric. The more generic F_β score applies additional weights, valuing one of precision or recall more than the other.

$$F_1 = \frac{2}{\text{recall}^{-1} + \text{precision}^{-1}} = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

CLASSIFICATION METRICS – THE CONFUSION MATRIX



https://www.researchgate.net/figure/Normalized-Confusion-Matrix-A-row-represents-an-instance-of-the-actual-class-ie-an_fig308074430_1

TIPS AND TRICKS FOR TRAINING CONVOLUTIONAL NEURAL NETWORKS (CNNs)

- Hyperparameters refer to the settings and configurations that are set before training the model. These parameters are not learned from the data but are manually defined by the user. Fine-tuning these hyperparameters can significantly impact the performance and training of the CNN model. Common hyperparameters are the **learning rate**, the **dropout rate** and the **weight decay**. It often requires experimentation and tuning to find the optimal combination of hyperparameters that results in the best model performance for a specific task or dataset.
- Data augmentation is a powerful technique to expand the training dataset by applying random transformations to the existing data. It helps in reducing overfitting and improves model generalization. Common data augmentation techniques for images include random rotations, translations, flips, zooms, and brightness adjustments. Applying these transformations increases the diversity of the training data, enabling the model to learn robust and invariant features.

TIPS AND TRICKS FOR TRAINING CONVOLUTIONAL NEURAL NETWORKS (CNNs)

- Regularization methods prevent overfitting and improve model generalization. Two widely used regularization techniques are L1 and L2 regularization, which add penalties to the loss function based on the magnitudes of the model's weights. This discourages large weight values and promotes simpler models. Another effective technique is dropout, which randomly sets a fraction of the neurons to zero during training, forcing the network to learn more robust and redundant representations.
- Learning Rate Schedules Optimizing the learning rate is crucial for successful training. Gradually reducing the learning rate over time (learning rate decay) or decreasing it when the validation loss plateaus (learning rate scheduling) can help the model converge to better solutions. Techniques like step decay, exponential decay, or cyclical learning rates can be employed to find an optimal learning rate schedule for the specific task.
- In CNNs, **normalization** is an approach employed to preprocess input data and guarantee uniform scaling of features. Its purpose is to facilitate quicker convergence during training, prevent numerical instabilities, and enhance the network's overall stability and performance.

OBJECT DETECTION: LOCALIZATION AND CLASSIFICATION

Object detection extends classification by requiring the model to provide spatial coordinates (bounding boxes) for multiple objects.

1. The Multi-Task Loss Function Object detection models must solve two problems simultaneously: Regression (where is the object?) and Classification (what is the object?). The total loss is a weighted sum:

$$\mathcal{L}_{total} = \lambda_{cls} \mathcal{L}_{cls} + \lambda_{box} \mathcal{L}_{box}$$

2. Intersection over Union (IoU) To evaluate the accuracy of a predicted box B_p against a ground truth box B_{gt} , we use the IoU metric:

$$IoU(B_p, B_{gt}) = \frac{\text{Area}(B_p \cap B_{gt})}{\text{Area}(B_p \cup B_{gt})}$$

3. Evolutionary Architectures: Two-Stage vs. One-Stage The field is generally split into two philosophies:

Feature	Two-Stage (e.g., Faster R-CNN)	One-Stage (e.g., YOLO, SSD)
Mechanism	Proposes regions, then classifies them.	Predicts boxes and classes in one pass.
Accuracy	Generally higher (better for small objects).	Slightly lower, but rapidly improving.
Speed	Slower (lower FPS).	Real-time capable (high FPS).



Task Output

Image classification

Car



Object

Detection

Car, exact bound-
ing box contain-
ing car

MODERN CONVOLUTIONAL DETECTION/SEGMENTATION

Detection

- R-FCN
- Faster R-CNN
- YOLO
- SSD

Segmentation

- Mask R-CNN
- SegNet
- U-Net, DeepLab, and more!

THE BACKBONE: RESIDUAL LEARNING AND GRADIENT FLOW

Before detecting objects, a model must extract robust features. The mathematical bottleneck in deep networks is the vanishing gradient.

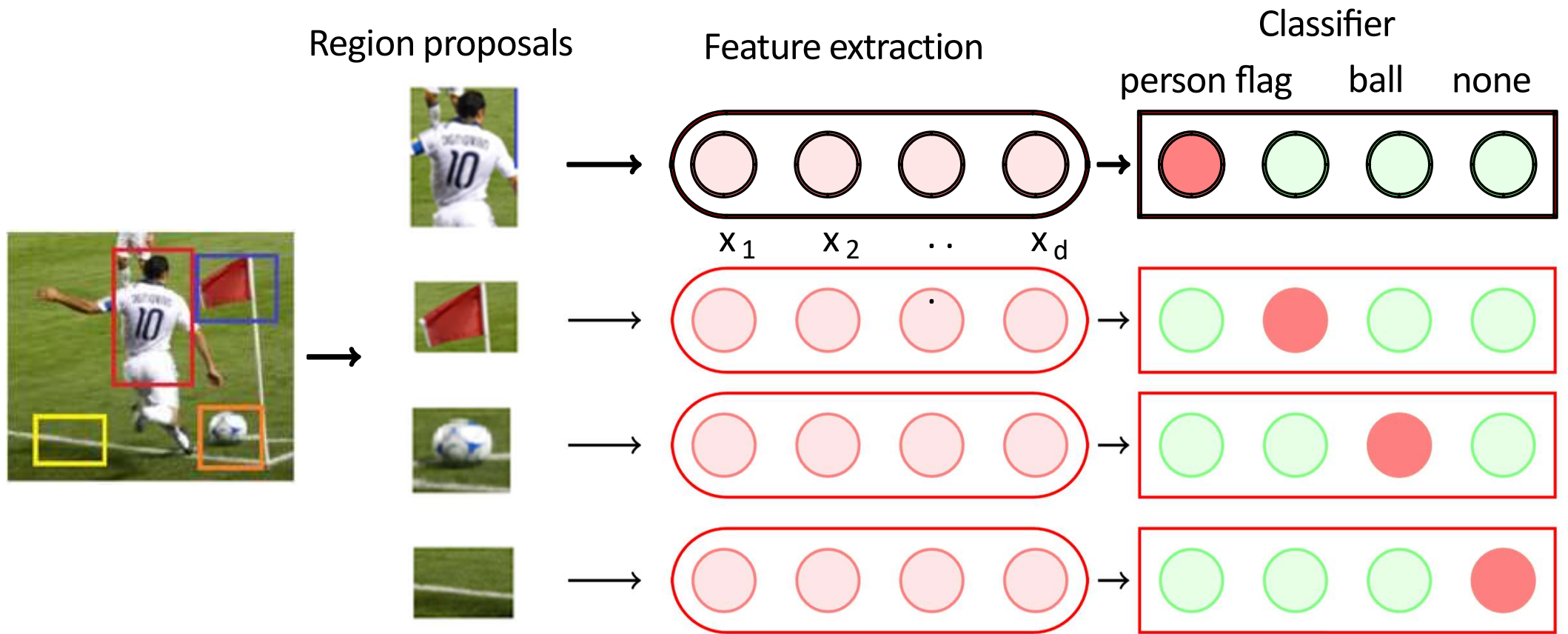
Analytic Justification for Skip Connections. In a standard plain network, the mapping $\mathcal{H}(\mathbf{x})$ is learned directly. In ResNet, we learn the residual mapping $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The output is:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

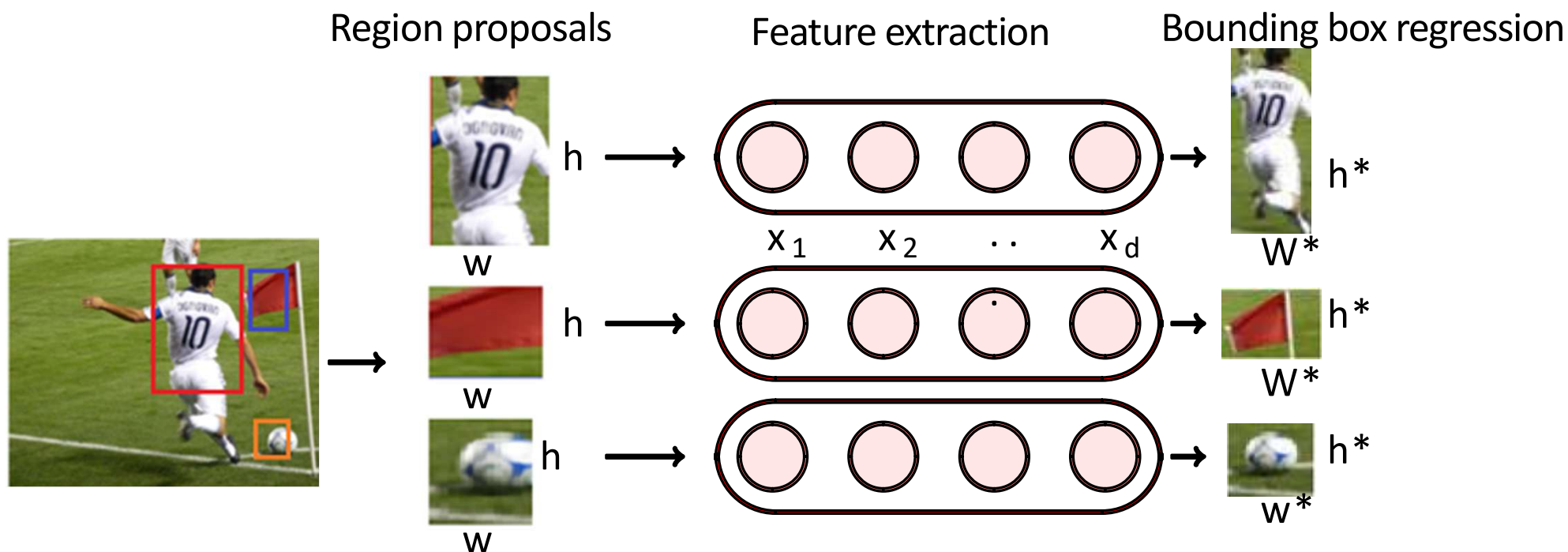
Mathematical Proof of Gradient Persistence: During backpropagation, the gradient of the loss $\mathcal{L}$ with respect to the input $\mathbf{x}_l$ at layer l is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \left(1 + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, W_i) \right)$$

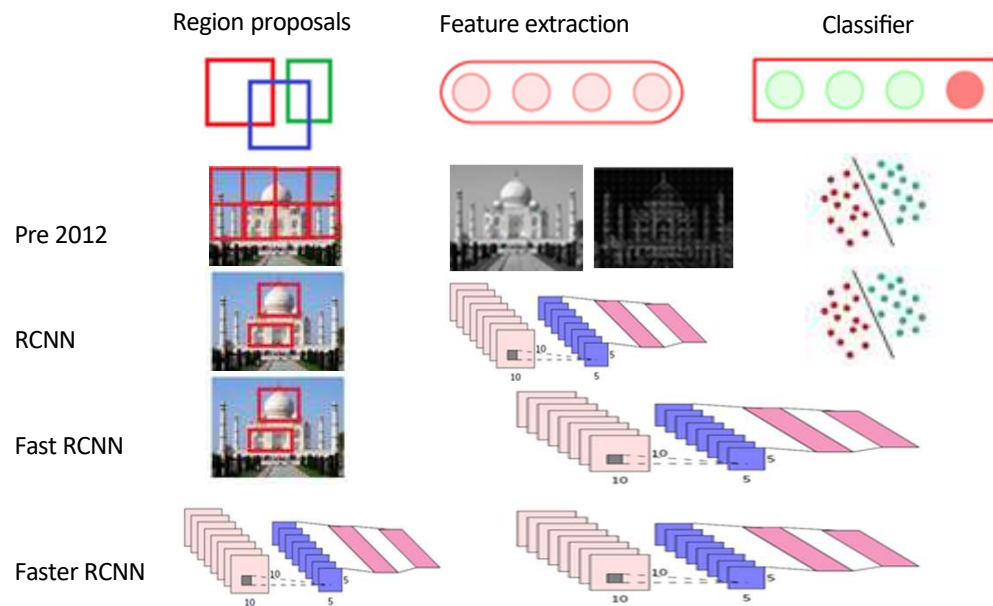
The term " $1 + \dots$ " ensures that the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{x}_L}$ can flow directly to any shallower layer l , even if the weights in $\mathcal{F}$ are very small.



Let us see a typical pipeline for object detection. It starts with a region proposal stage where we identify potential regions which may contain objects.

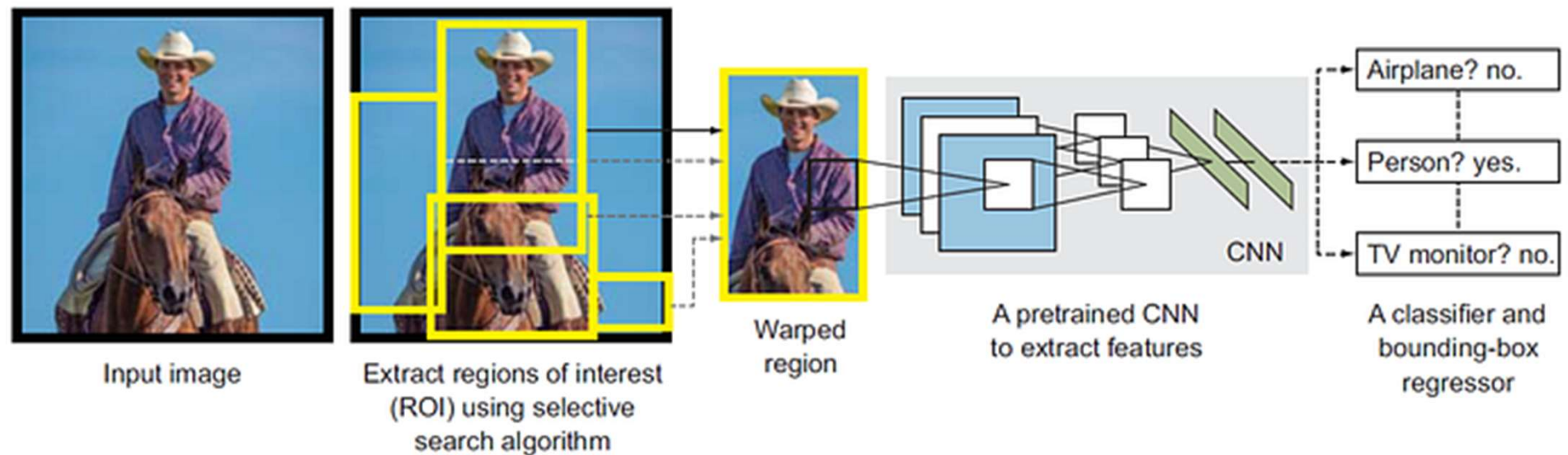


In addition, we would also like to correct the proposed bounding boxes This is posed as a regression problem (for example, we would like to predict w^* , h^* from the proposed w and h)



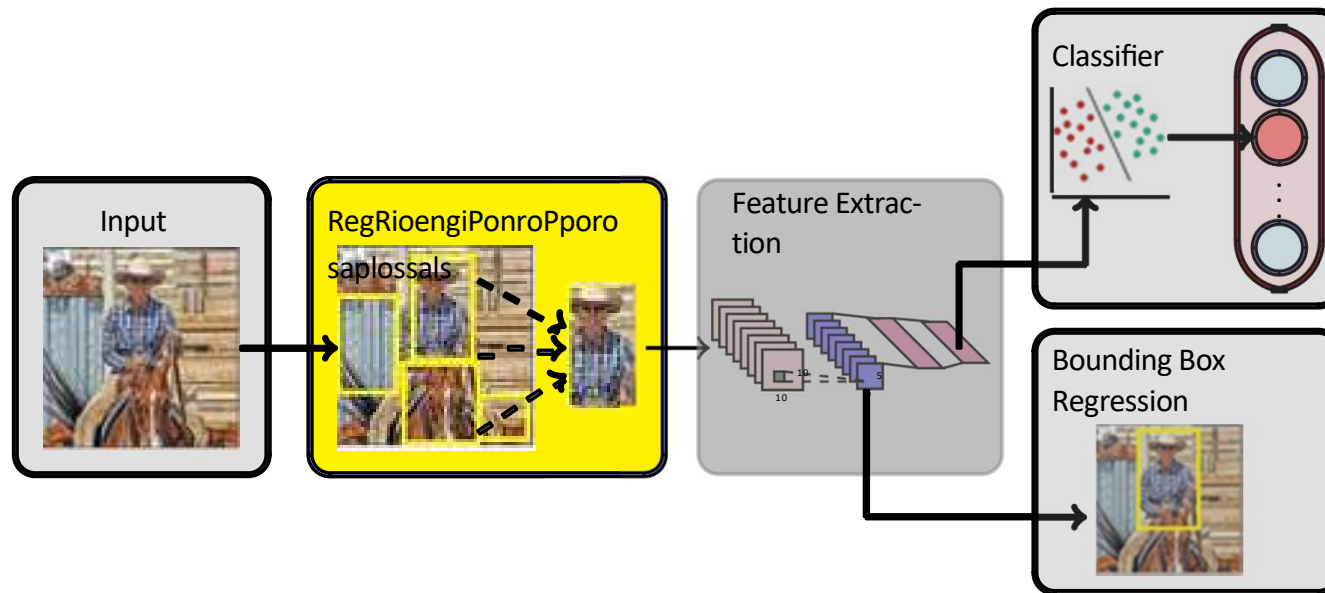
- Propose all possible regions in the image of varying sizes (almost brute force).
 - Use handcrafted features (SIFT, HOG)
 - Train a linear classifier using these features
-

RCNN MODEL FOR OBJECT DETECTION



COMPONENTS OF R-CNN

- Extract region proposals : These regions have a high probability of containing an object. A selective search algorithm scans the input image to find regions, and proposes them as RoI to be processed by the next modules. The proposed RoIs are then warped to have a fixed size because they usually vary in size, but CNN requires a fixed input image size.
- Feature extraction module : A pretrained convolutional network is run to extract features from each ROI.
- Classification module : A classifier like SVM is used to classify candidate detections. SVM determines whether the region is an object or not and, if so, what kind of object it is. This is based on the extracted features from the previous step.
- Localization module : The location and size of the bounding box that surrounds the object are predicted. The bounding box is represented by identifying four values: the x and y coordinates of the box's origin (x, y), the width, and the height of the box (w, h). Putting this together, the regressor predicts the four real-valued numbers that define the bounding box as the following tuple: (x, y, w, h).

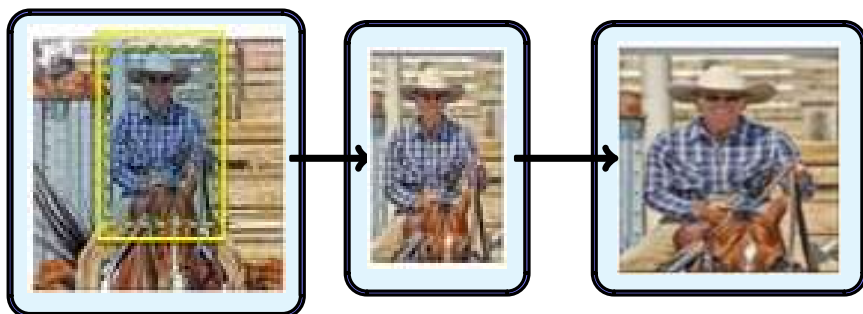
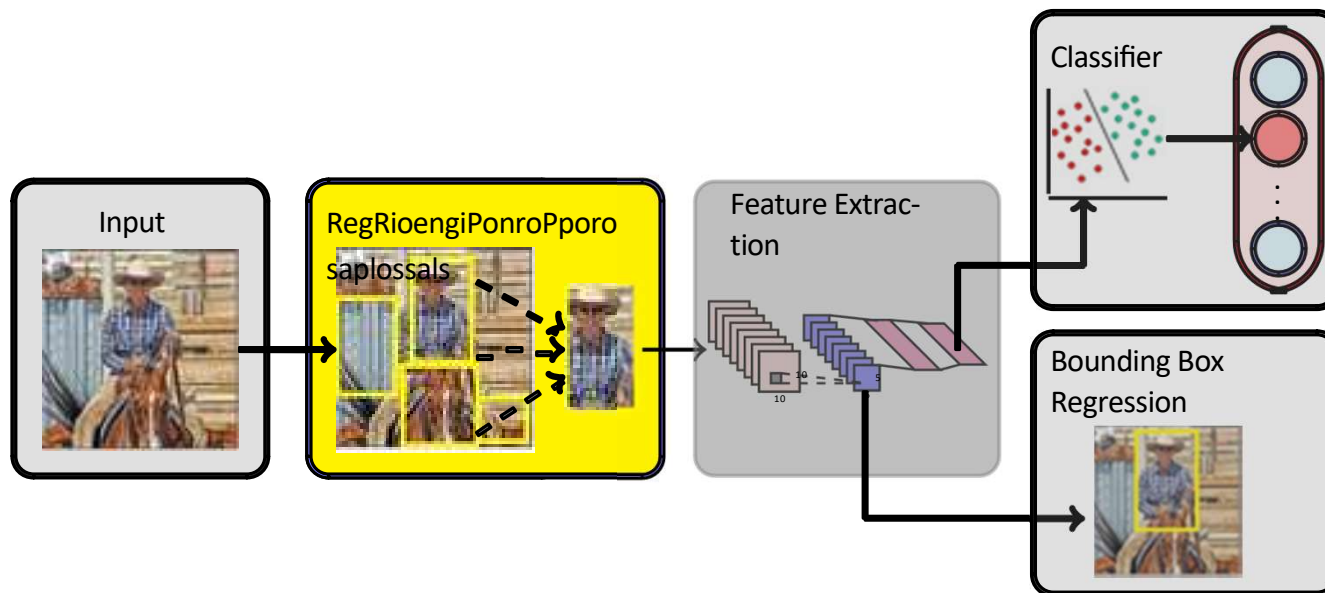


- Selective Search for region proposals
- Hierarchical clustering at different scales
- Such a hierarchical clustering finds different objects at different scales

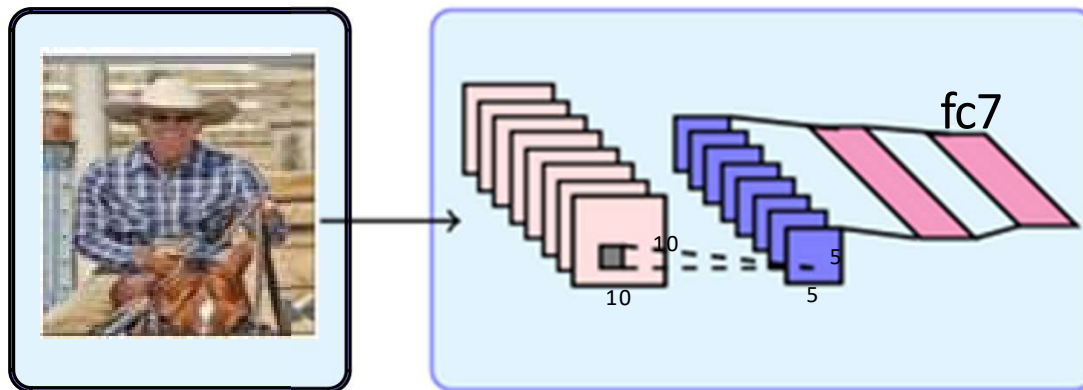
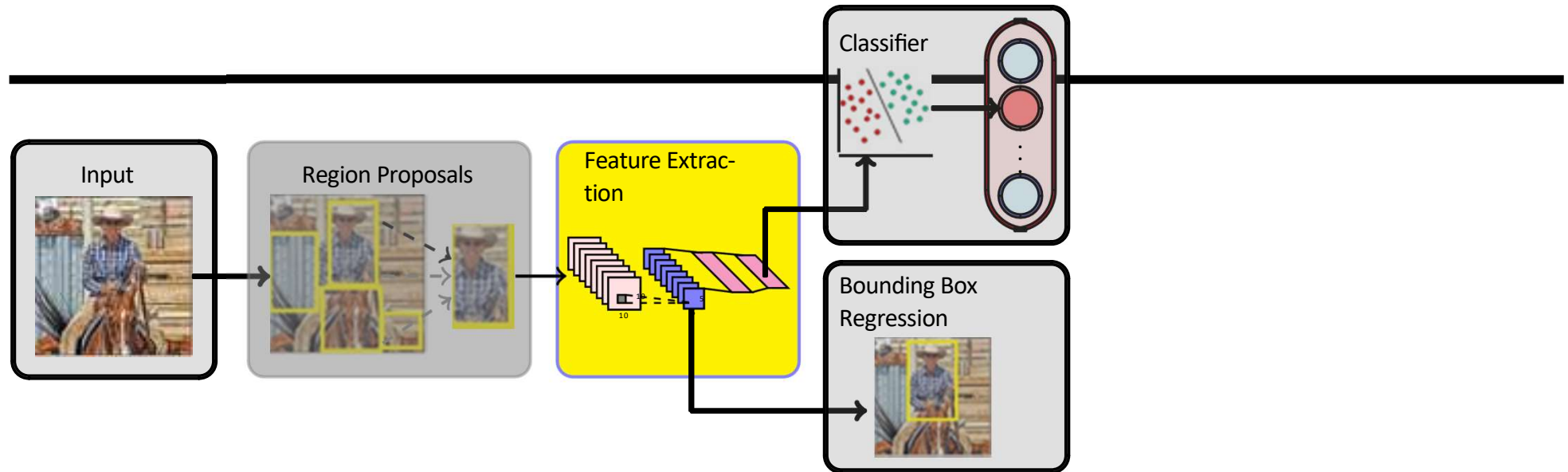


SELECTIVE SEARCH REGION PROPOSAL

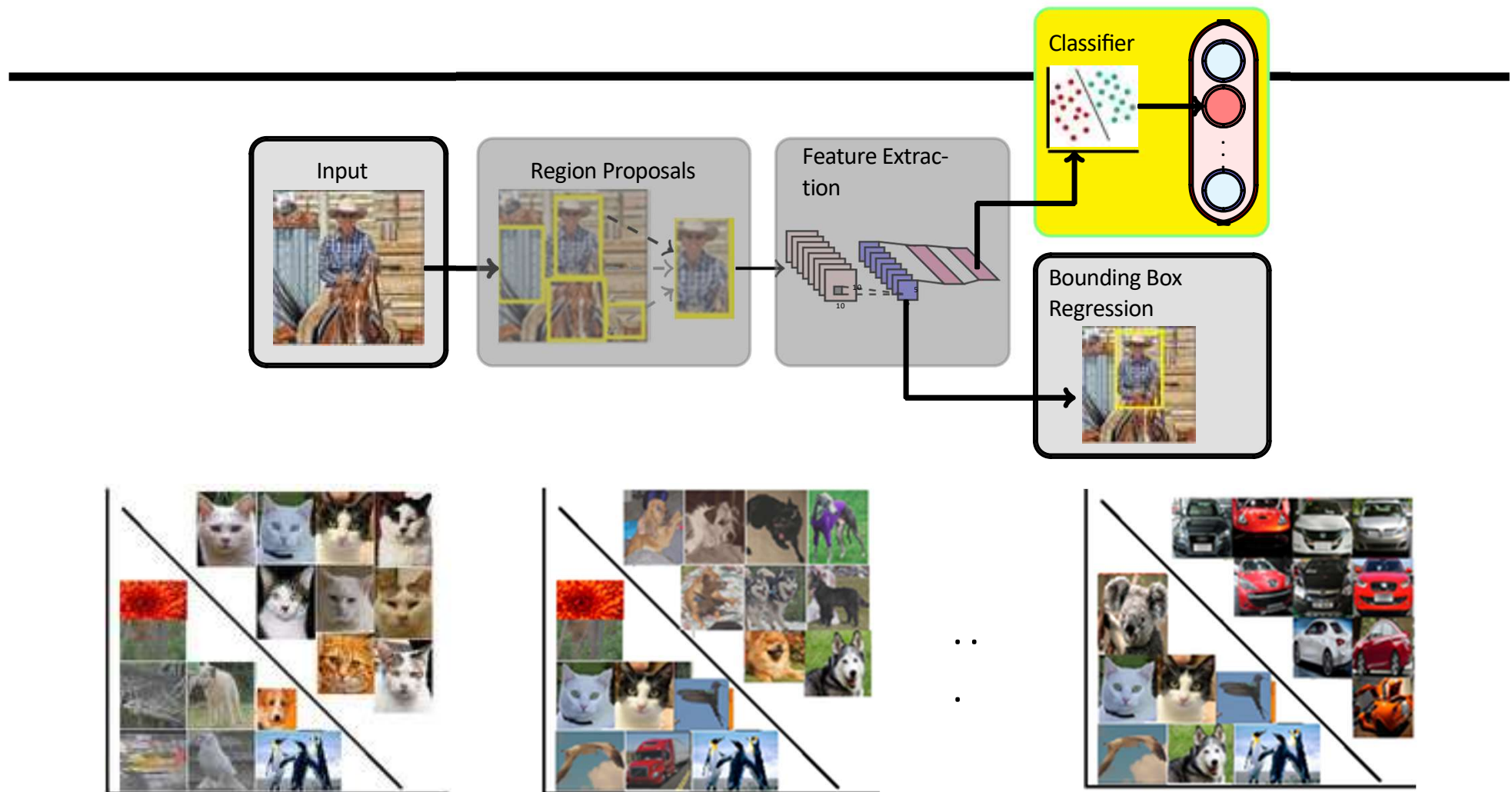
- A traditional, non-learning-based, greedy process that works in a bottom-up hierarchical manner:
 - **Initial Segmentation:** The image is first divided into many small, initial regions (superpixels) based on pixel intensities using a graph-based segmentation method.
 - **Hierarchical Grouping:** The algorithm then iteratively groups the two most similar adjacent regions into a single, larger region. This process is repeated, forming a hierarchy of segments at different scales.
 - **Similarity Measures:** Similarity between regions is calculated using a combination of various factors to ensure the generated regions are likely to be complete objects, including:
 - Color
 - Texture
 - Size (encourages smaller regions to merge early, ensuring coverage at all scales)
 - Shape compatibility (measures how well regions fit into one another to avoid gaps)
 - **Region Proposals:** All the initial small regions and the larger merged regions generated during the iterative process are collected as the final set of region proposals. A bounding box is placed around each one.
-



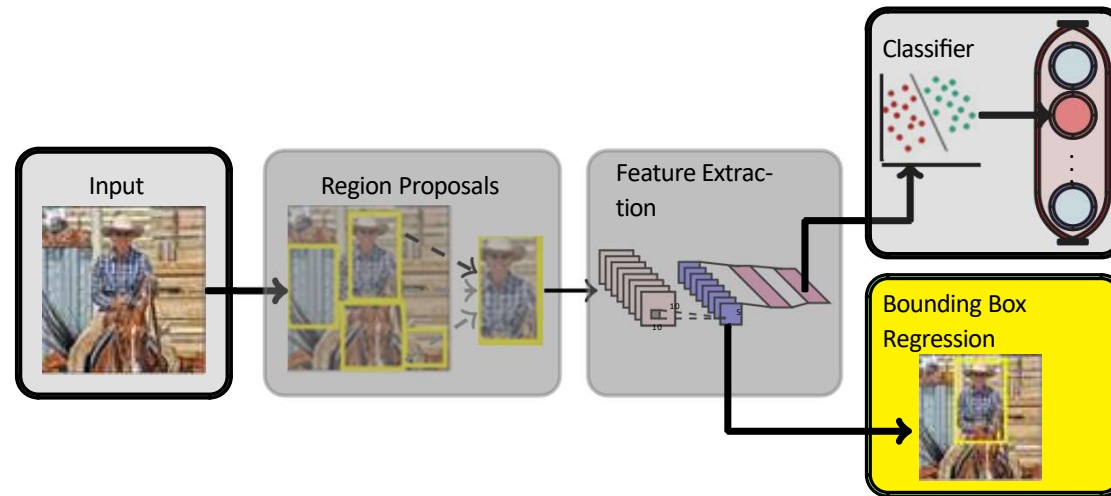
Proposed regions are cropped to form mini images
Each mini image is scaled to match the CNN's
(feature extractor) input size



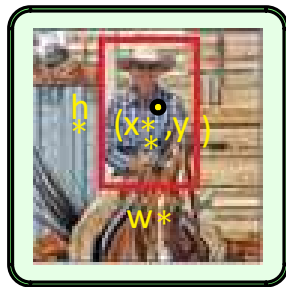
- For feature extraction any CNN trained for Image Classification can be used (AlexNet/ VGGNet etc.)
- Outputs from fc7 layer are taken as features
- CNN is fine tuned using ground truth (cropped) object images



Linear models (SVMs) are used for classification (1 model per class)



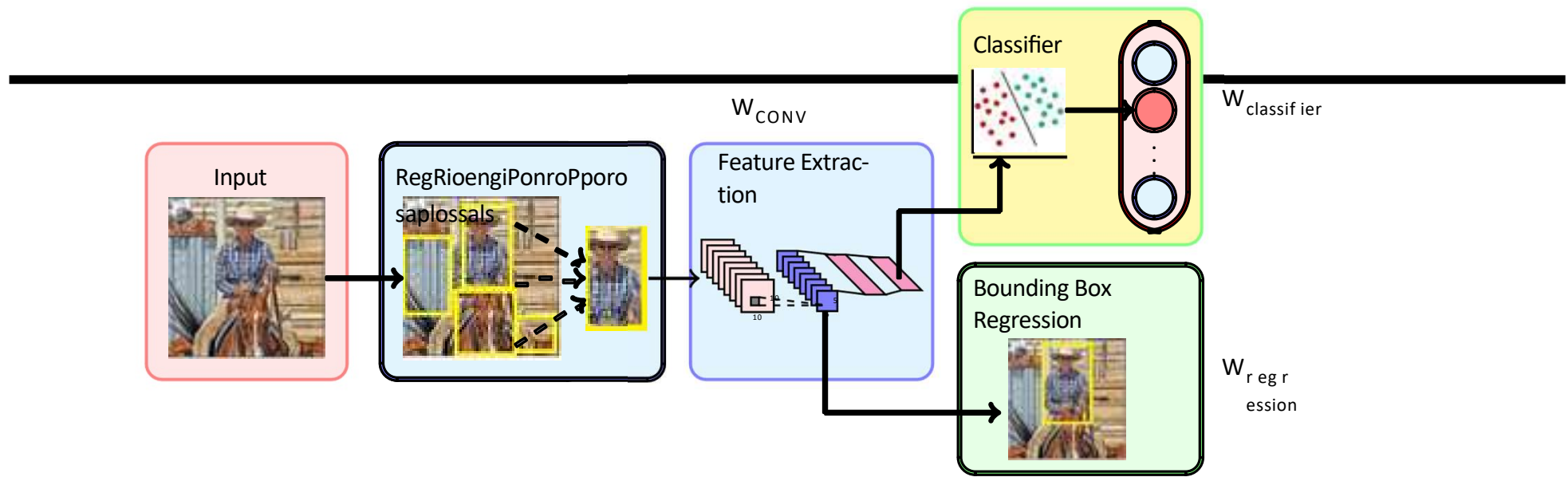
ProposedBox



TrueBox

z : features from pool5 layer of the network

The proposed regions may not be perfect
 Learn four regression models which will learn to predict
 x^*, y^*, w^*, h^*

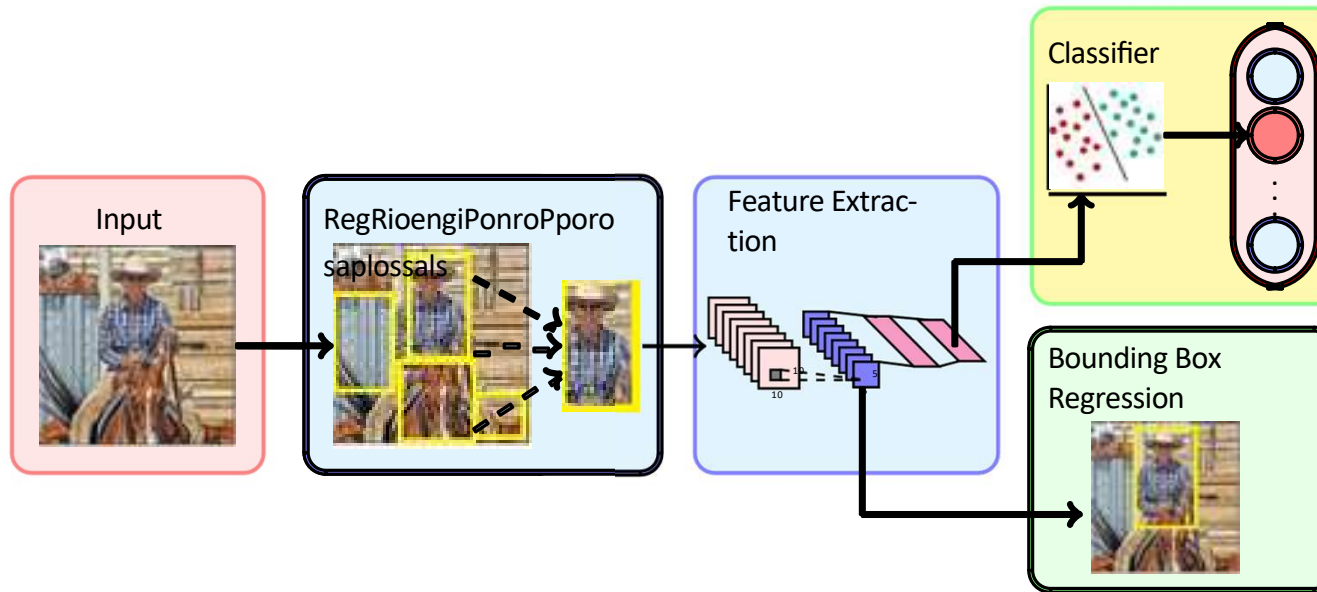


W_{CONV} is taken as it is from a CNN trained for Image classification (say on ImageNet)

W_{CONV} is then fine tuned using ground truth (cropped) object images

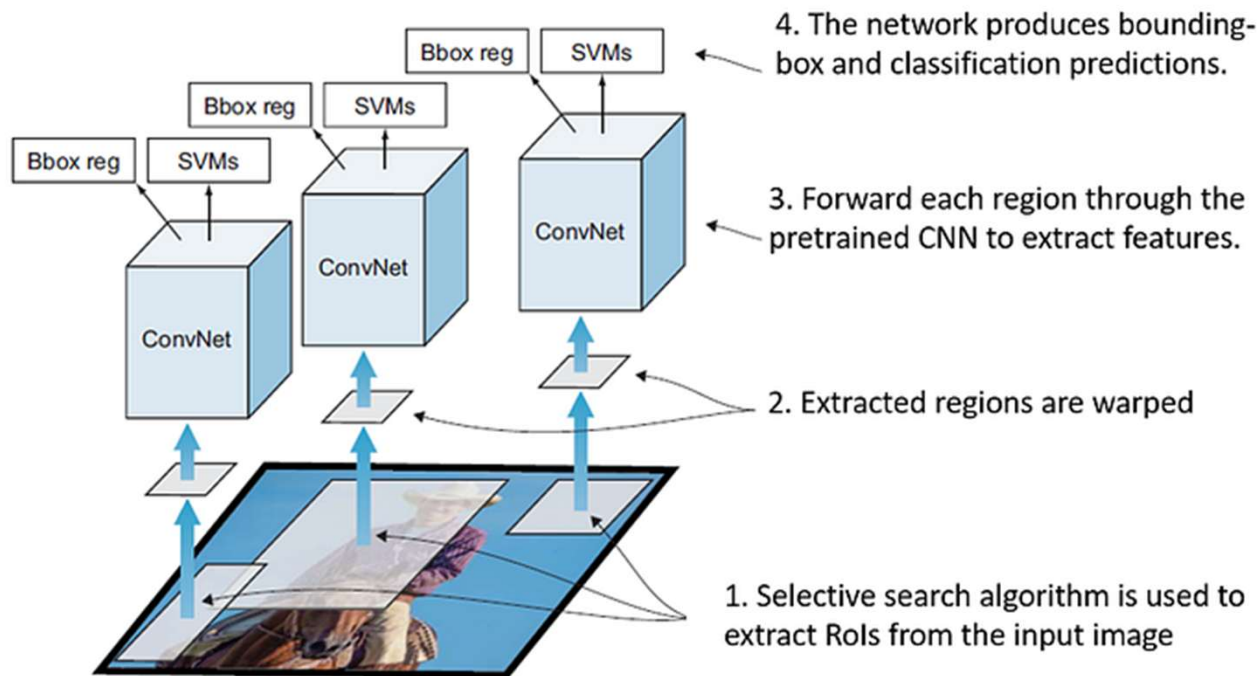
$W_{classifier}$ is learned using ground truth (cropped) object images

$W_{regression}$ is learned using ground truth bounding boxes



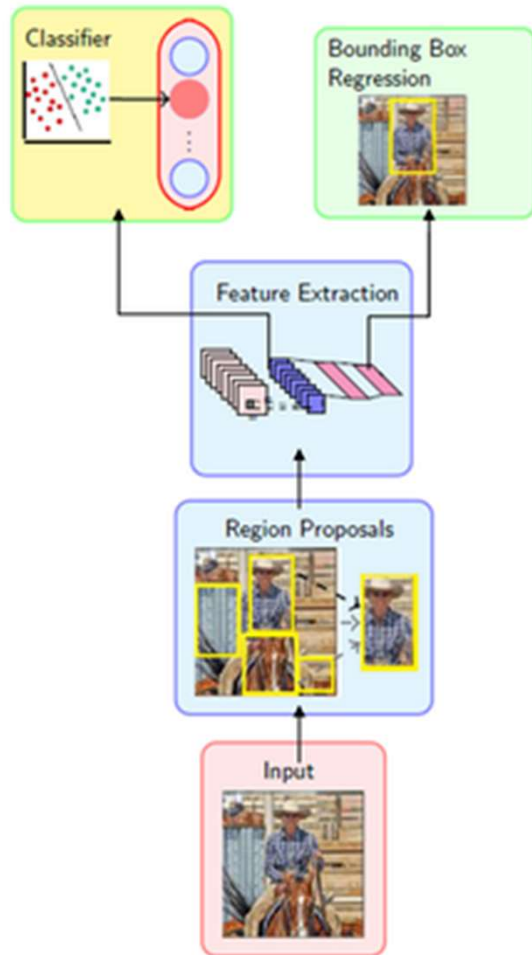
The computational cost for processing one image at test time:

$$\text{Inference Time} = \text{Proposal Time} + \# \text{ Proposals} \times \text{Convolution Time} + \# \text{ Proposals} \times \text{classification} + \# \text{ Proposals} \times \text{regression}$$



On average selective search gives 2K region proposal

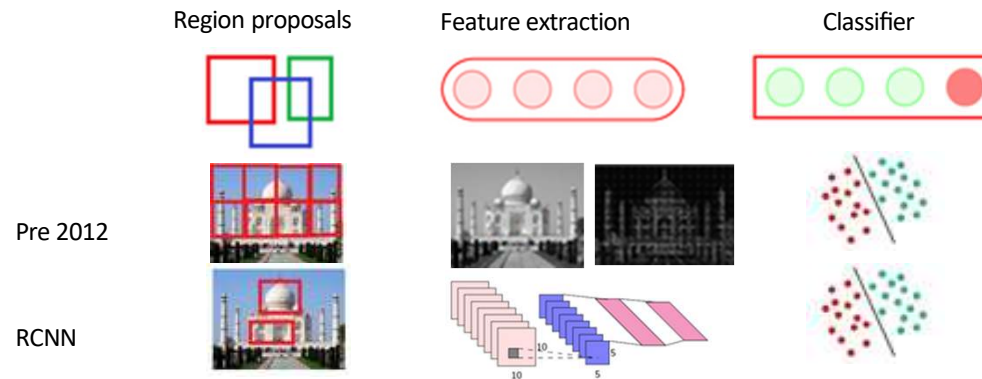
Each of these pass through the CNN for feature extraction followed by classification and regression



No joint learning Use ad hoc training objectives

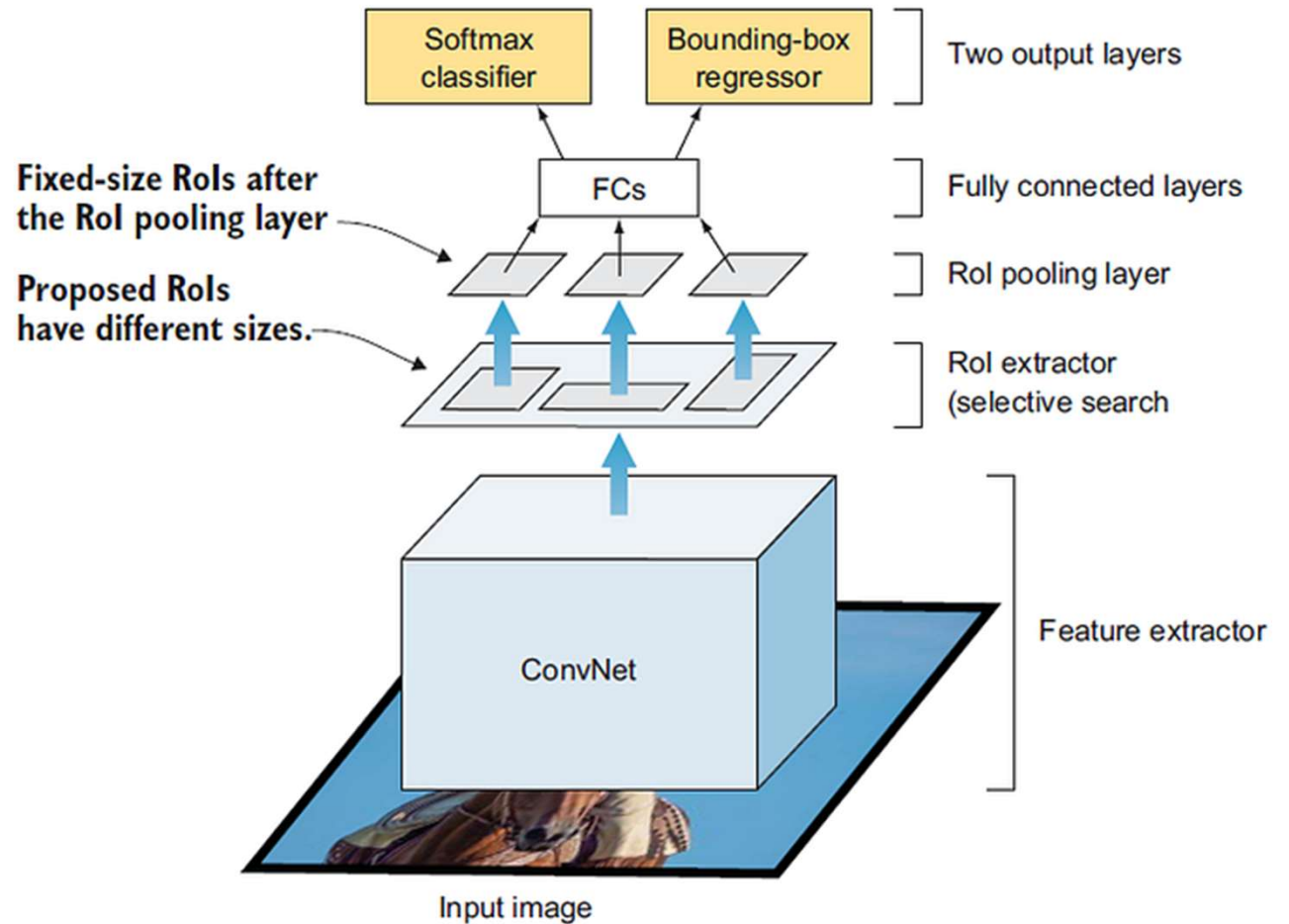
- Fine tune network with softmax classifier (log loss)
- Train post-hoc linear SVMs (hinge loss)
- Train post-hoc bounding-box regressors (squared loss)

Training (≈ 3 days) and testing (47s per image) is slow¹. Takes a lot of disk space



- Region Proposals: Selective Search
 - Feature Extraction: CNNs
 - Classifier: Linear
-

FAST RCNN MODEL FOR OBJECT DETECTION



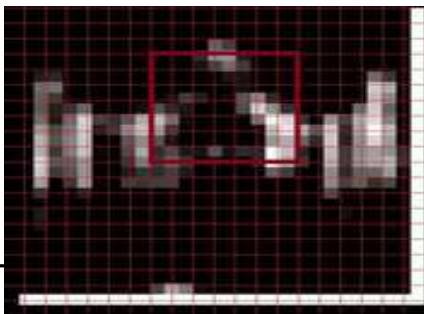
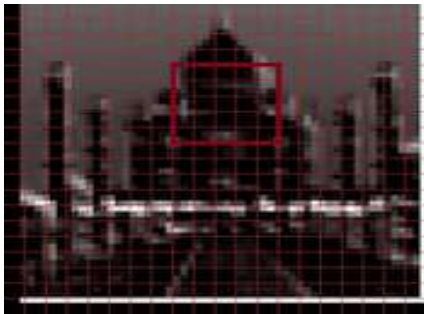
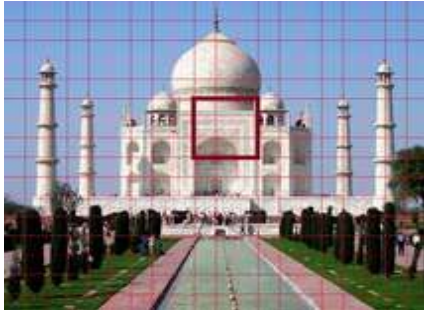
KEY DIFFERENCE FROM R-CNN

Fast R-CNN produces region proposals based on the last feature map of CNN, not from the original input image like R-CNN. In other words, Fast R-CNN implements CNN first and then generates region proposals. On the contrary, R-CNN produces region proposals first and then implements CNN.

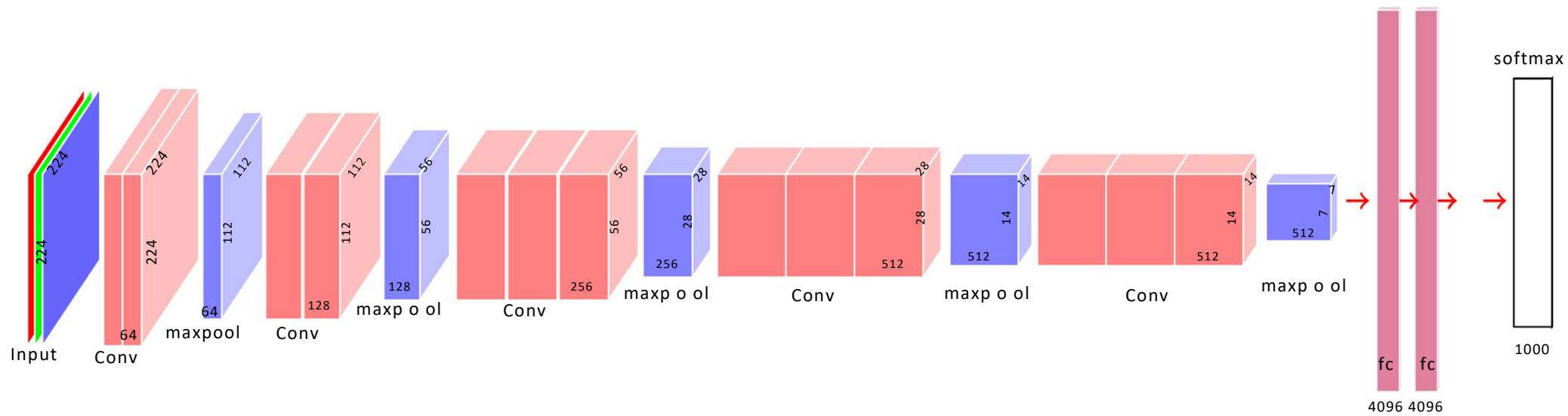
FAST R-CNN ARCHITECTURE

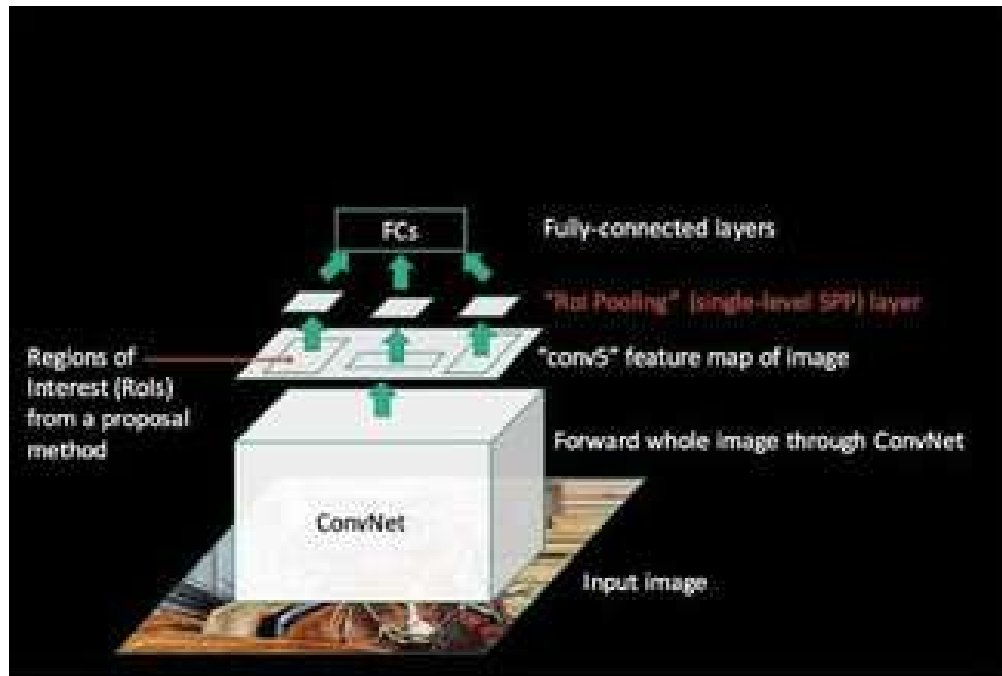
- **Feature extractor module** : The network starts with CNN to extract features from the full image.
 - **RoI extractor** : The selective search algorithm proposes region candidates per image.
 - **RoI pooling layer** : This is a new component that was introduced to extract a fixed-size window from the feature map before sending the RoIs to the fully connected layers. It uses max pooling to convert the features inside any valid RoI into a small feature map with a fixed spatial extent of height $\times$ width ($H \times W$). In shorts, the pooling layer is applied on the last feature map layer extracted from the CNN, and its goal is to extract fixed-size RoIs to feed to the fully connected layers and then the output layers.
 - **Two-head output layer** : The model branches into two heads. A softmax classifier layer that outputs a discrete probability distribution per RoI. A bounding-box regressor layer to predict offsets relative to the original RoI.
-

RECEPTIVE FIELD

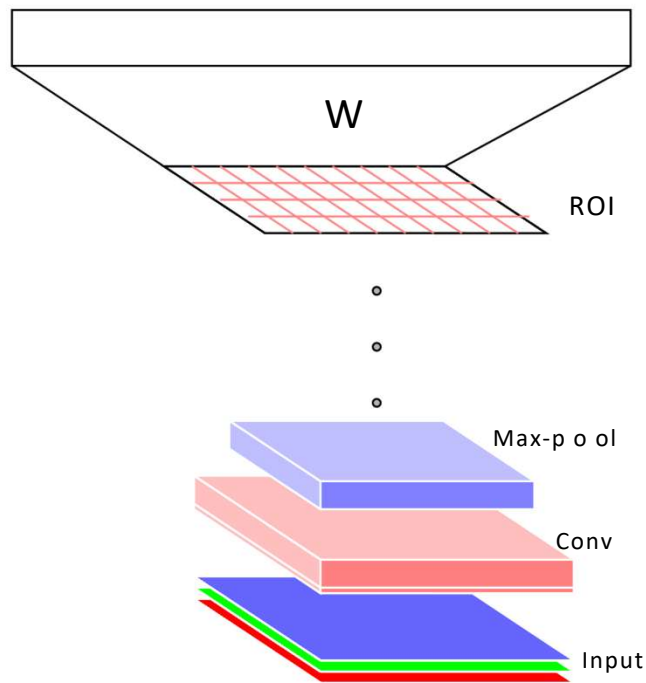


- Suppose we apply a 3×3 kernel to an image
 - What is the region of influence of each pixel in the resulting output ?
 - Each pixel contributes to a 5×5 region
Suppose we again apply a 3×3 kernel on this output?
 - What is the region of influence of the original pixel from the input ? (a 7×7 region)
-

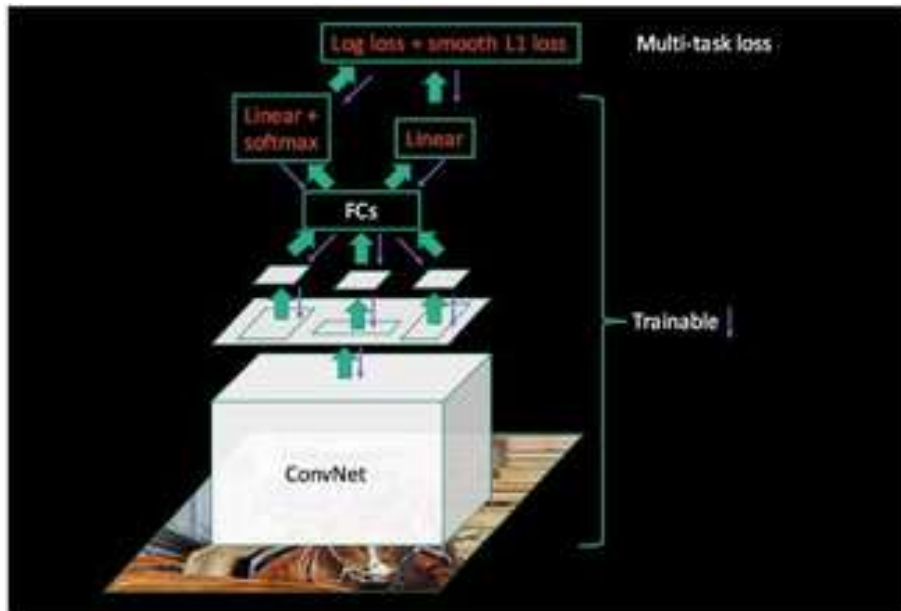




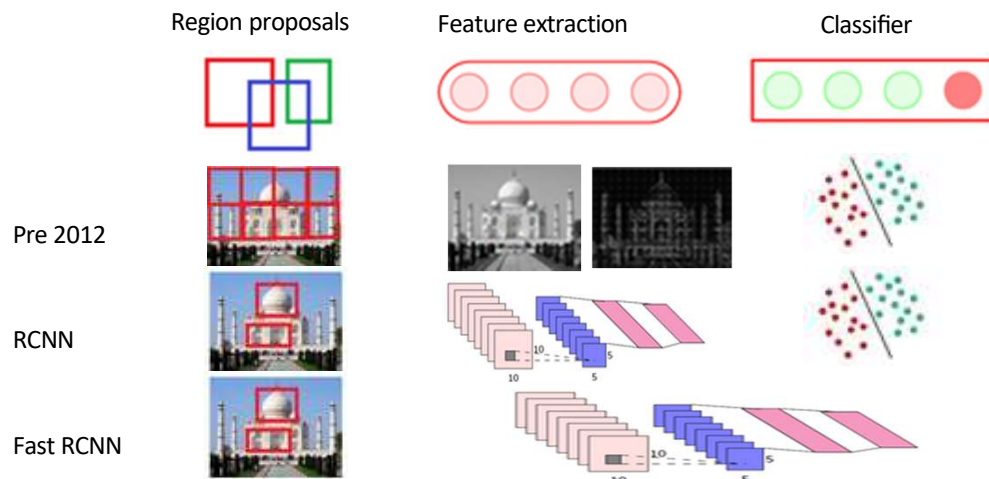
- Using this idea we could get a bounding box's region of influence on any layer in the CNN
- The projected Region of Interest (RoI) may be of different sizes
- Divide them into k equally sized regions of dimension $H \times W$ and do max pooling in each of those regions to construct a k dimensional vector
- Connect the k dimensional vector to a fully connected layer
- This max pooling operation is called RoI pooling



- Recall that the last pooling layer of VGGNet-16 results in an output of size $512 \times 7 \times 7$
 - We replace the last max pooling layer by a RoI pooling layer
 - We set $H = W = 7$ and divide each of these RoIs into ($k = 49$) regions
 - We do this for every feature map resulting in an output of size 512×49
 - This output is of the same size as the output of the original max pooling layer
 - It is thus compatible with the dimensions of the weight matrix connecting the original pooling layer to the first
-

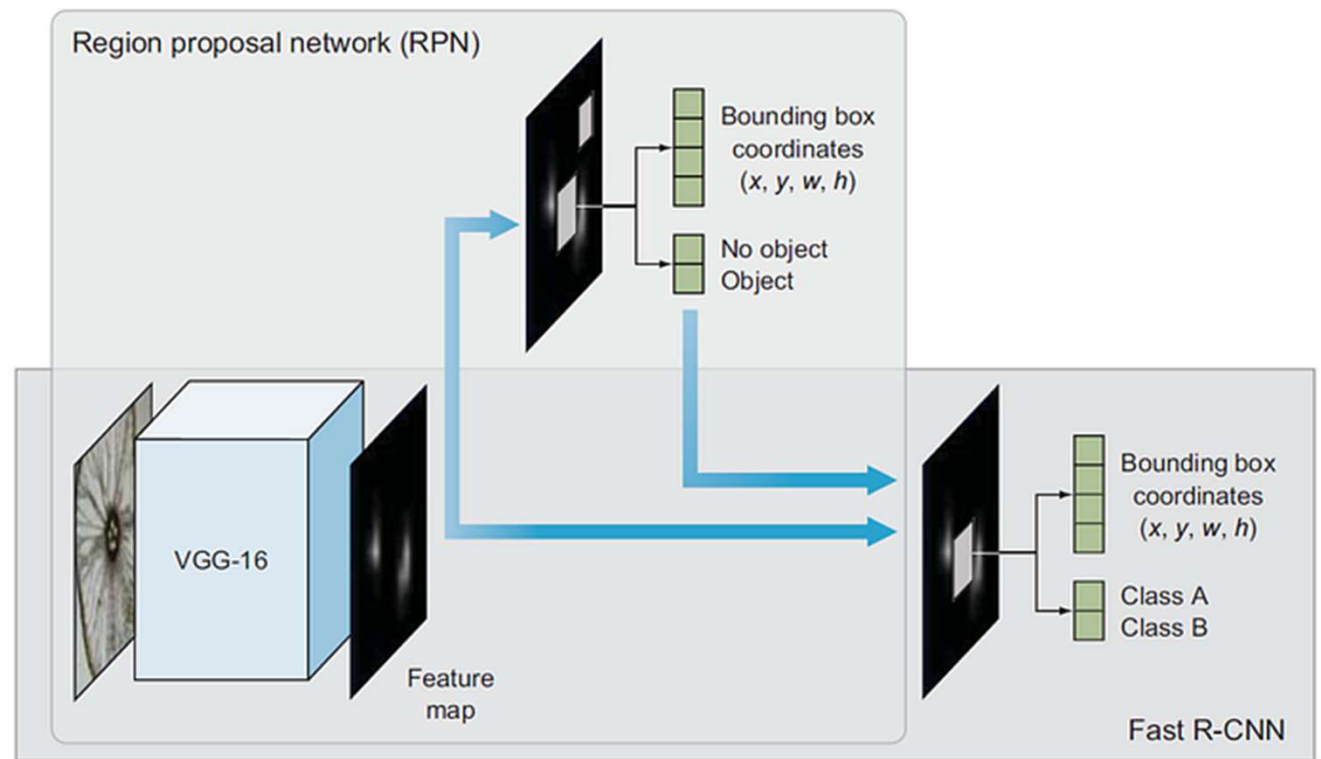


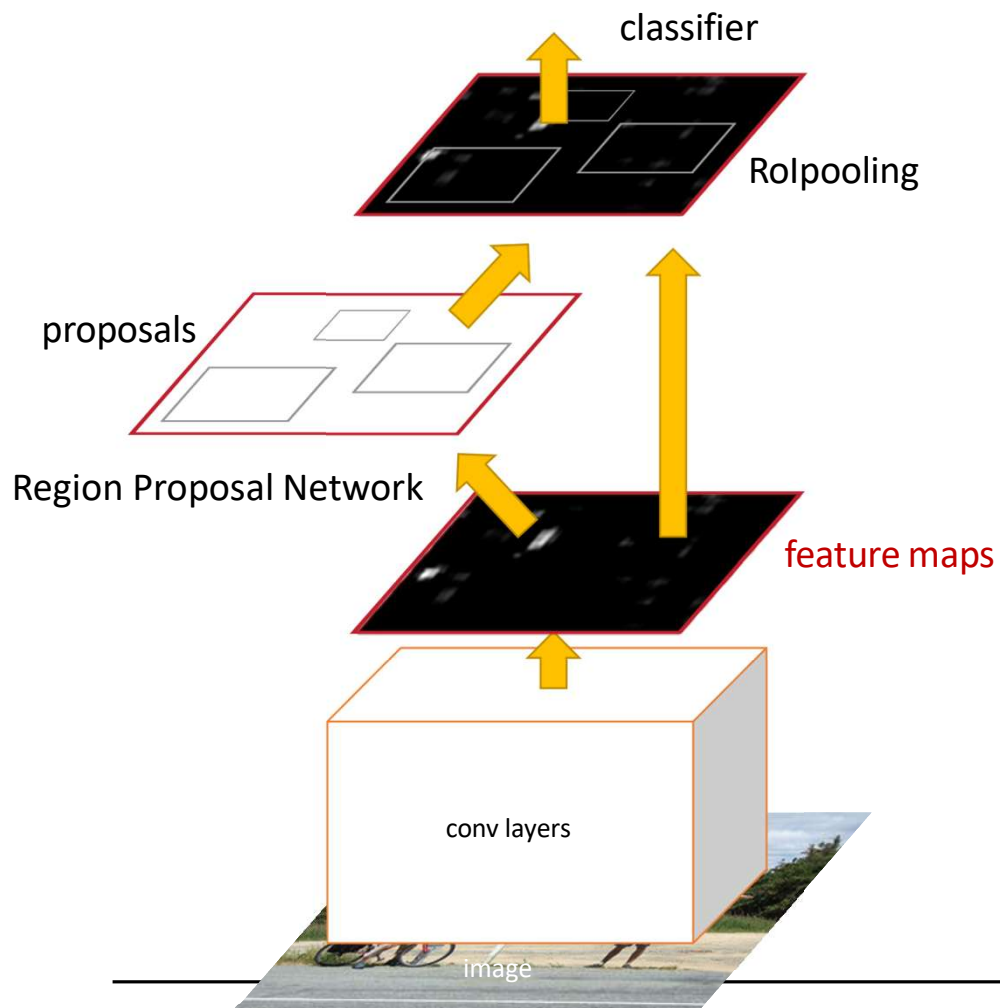
- Once we have the FC layer it gives us the representation of this region proposal
- We can then add a softmax layer on top of it to compute a probability distribution over the possible object classes
- Similarly we can add a regression layer on top of it to predict the new bounding box (w^*, h^*, x^*, y^*)



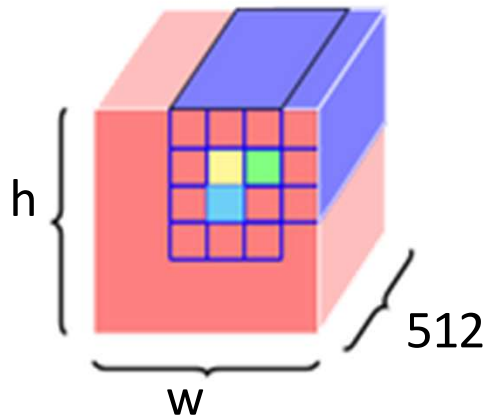
- Region Proposals: Selective
 - Feature Extraction: CNN
 - Classifier: CNN
-

FASTER RCNN MODEL FOR OBJECT DETECTION

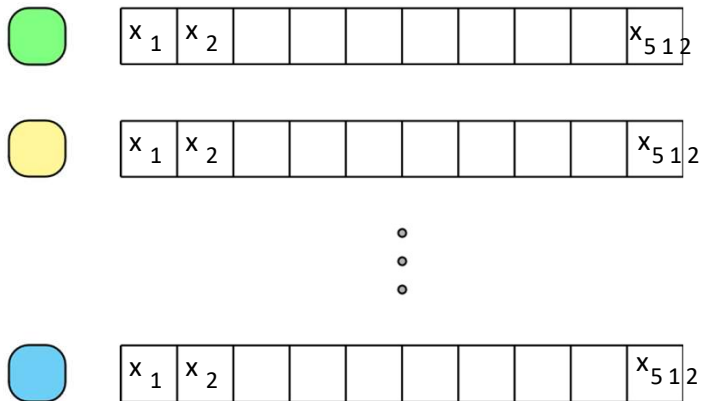




- So far the region proposals were being made using Selective Search algorithm
 - Idea: Can we use a CNN for making region proposals?
 - How? Well it's slightly tricky
 - We will illustrate this using VGGNet
-

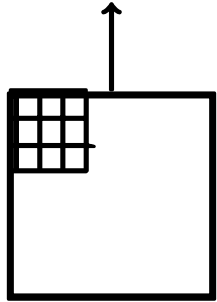


- Consider the output of the last convolutional layer of VGGNet
- Now consider one cell in one of the 512 feature maps
- If we apply a 3×3 kernel around this cell then we will get a 1D representation for this cell
- If we repeat this for all the 512 feature maps then we will get a 512 dimensional representation for this position
- We use this process to get a 512 dimensional representation for each of the $w \times h$ positions

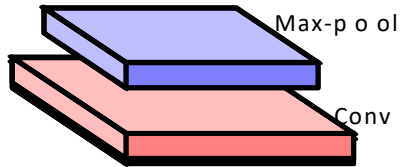




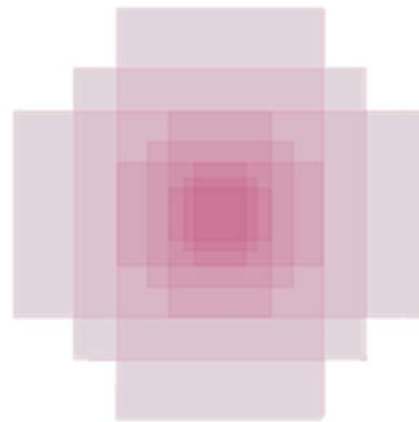
1×2 512



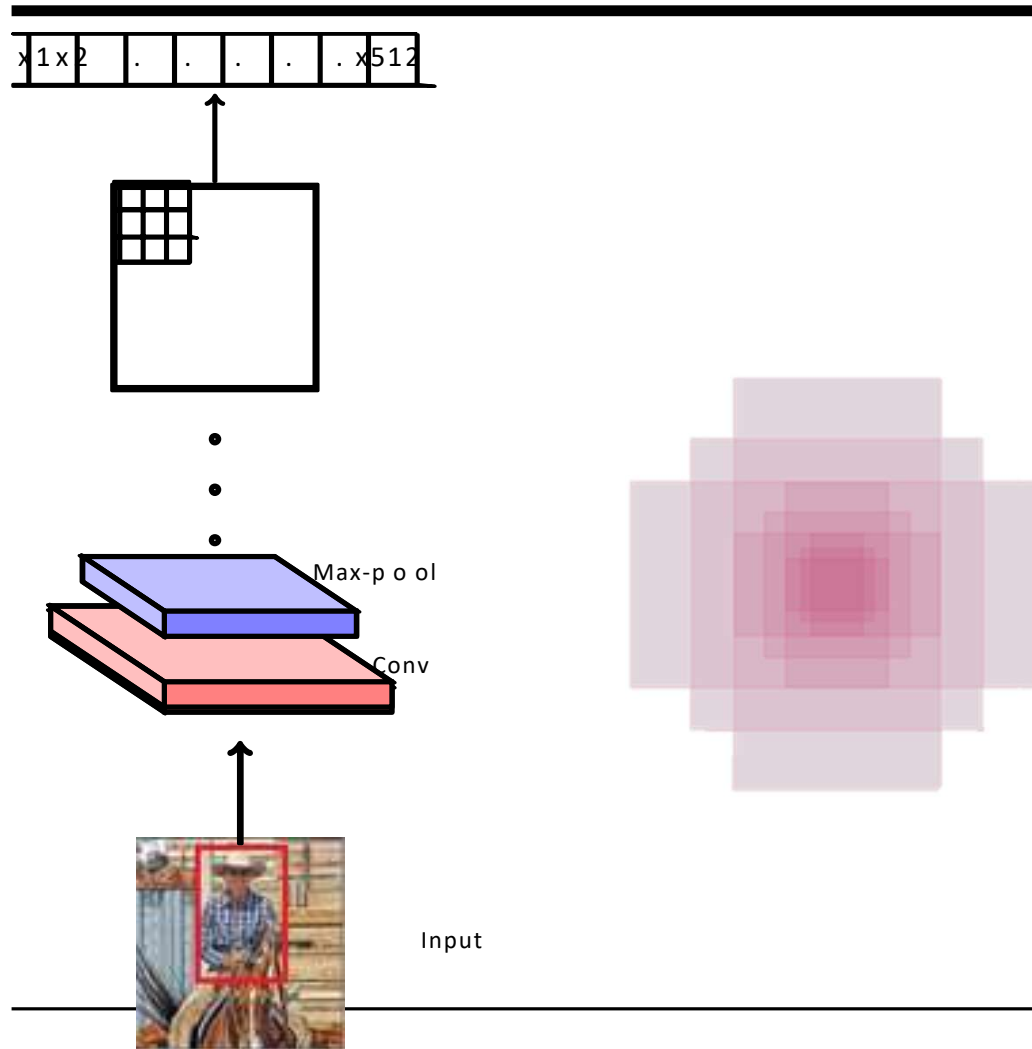
⋮



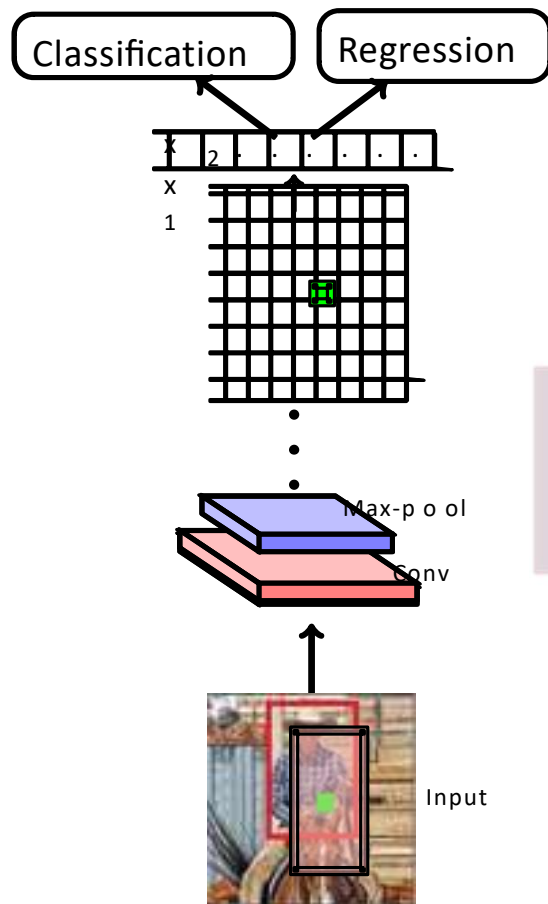
Input



- We now consider k bounding boxes (called anchor boxes) of different sizes & aspect ratio
- We are interested in the following two questions:
- Given the 512d representation of a position, what is the probability that a given anchor box centered at this position contains an object? (Classification)
- How do you predict the true bounding box from this anchor box? (Regression)



- We train a classification model and a regression model to address these two questions
- How do we get the ground truth data?
- What is the objective function used for training?



- Consider a ground truth object and its corresponding bounding box
 - Consider the projection of this image onto the conv5 layer
 - Consider one such cell in the output
 - This cell corresponds to a patch in the original image
 - Consider the center of this patch
 - We consider anchor boxes of different sizes
 - For each of these anchor boxes, we would want the classifier to predict 1 if this anchor box has a reasonable overlap ($\text{IoU} > 0.7$) with the true grounding box
-

The full network is trained using the following objective.

$$L(p_i, t_i) = \frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p^*_i) + \frac{\lambda}{N_{reg}} \sum_i p^*_i L_{reg}(t_i, t^*_i)$$

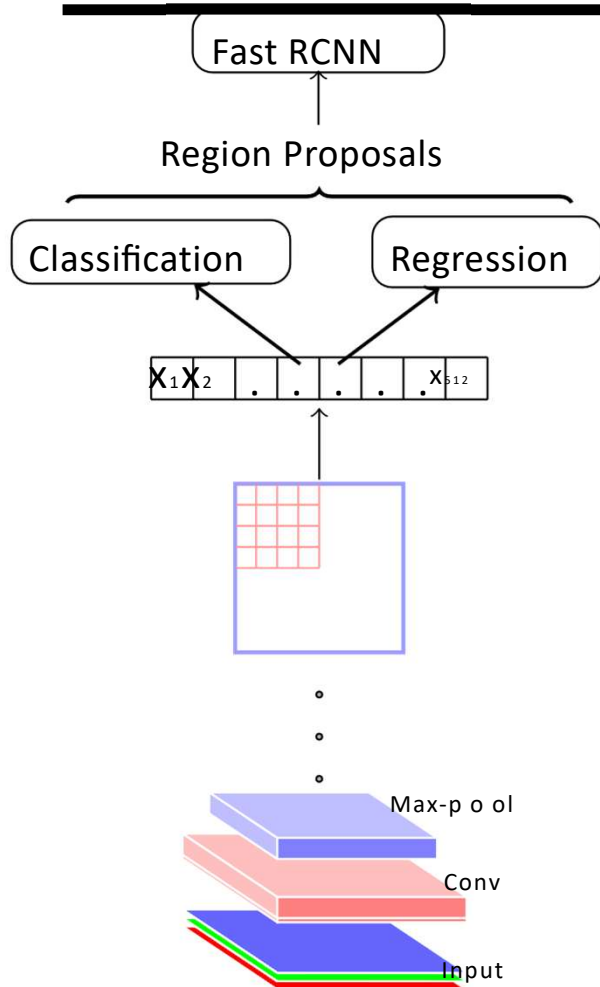
$$p^*_i = \begin{cases} 1, & \text{if anchor box contains ground truth object} \\ 0, & \text{otherwise} \end{cases}$$

p_i = predicted probability of anchor box containing an object

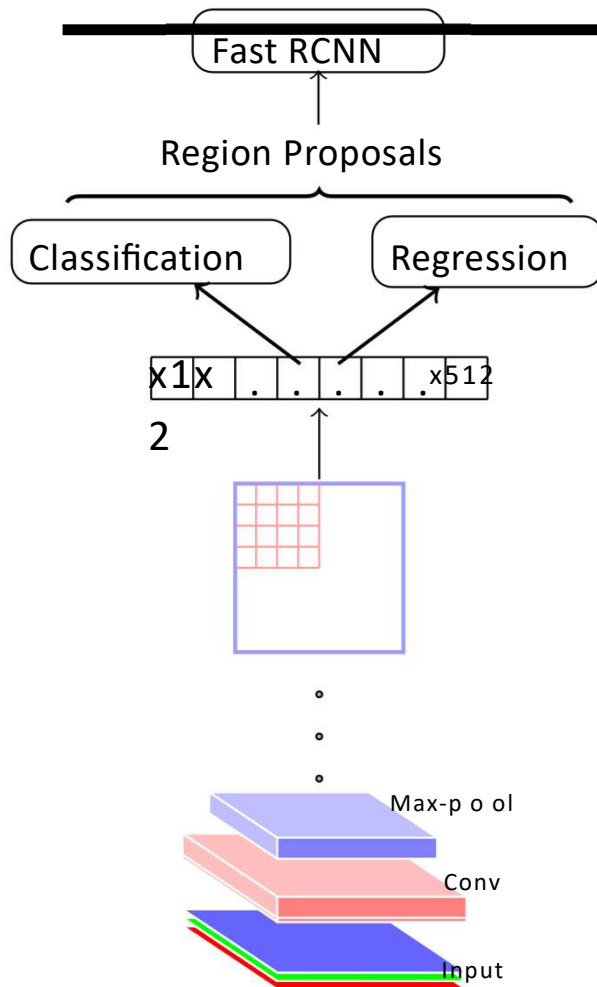
N_{cls} = batch-size

N_{reg} = batch-size $\times k$

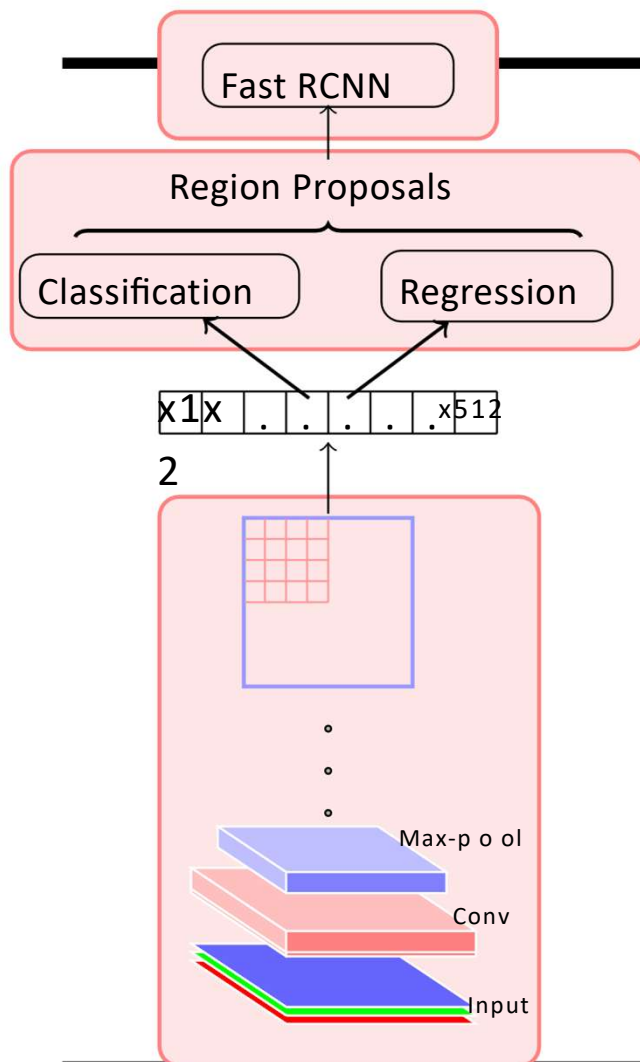
k = anchor boxes



- So far we have seen a CNN based approach for region proposals instead of using selective search
- We can now take these region proposals and then add fast RCNN on top of it to predict the class of the object
- And regress the proposed bounding box



- But the fast RCNN would again use a VGG Net
- Can't we use a single VGG Net and share the parameters of RPN and RCNN?



Faster RCNN: Training

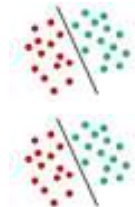
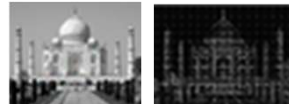
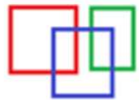
- Fine-tune RPN using a pre-trained ImageNet network
- Fine-tune fast RCNN from a pre-trained ImageNet network using bounding boxes from step 1
- Keeping common convolutional layer parameters fixed from step 2, fine-tune RPN (post conv5 layers)
- Keeping common convolution layer parameters fixed from step 3, fine-tune fc layers of fast RCNN

Region proposals

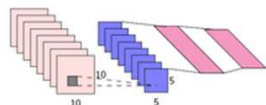
Feature extraction

Classifier

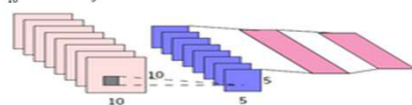
Pre 2012



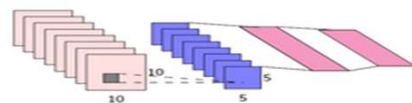
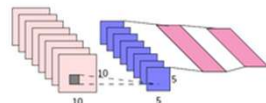
RCNN



Fast RCNN



Faster RCNN

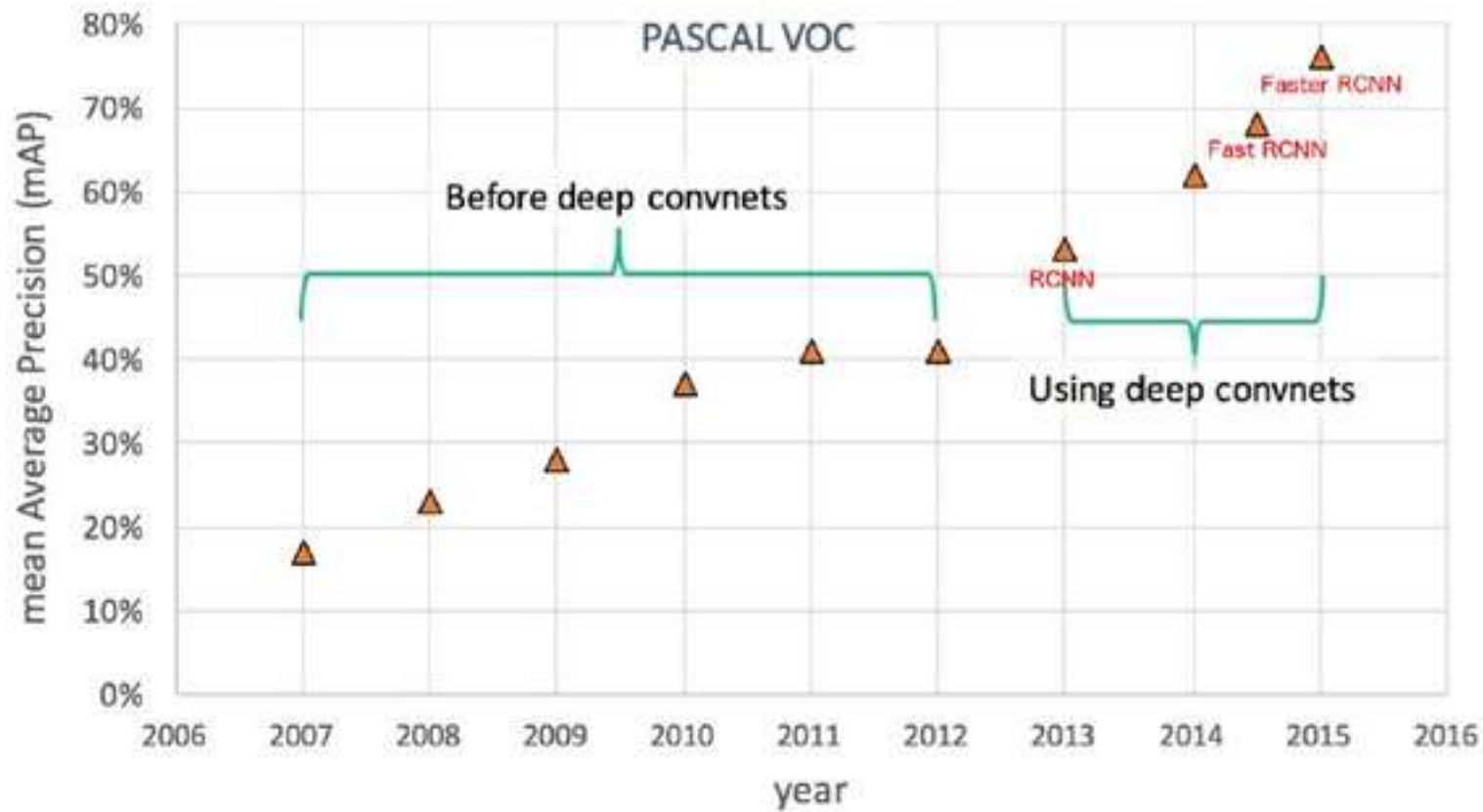


- Region Proposals: CNN
- Feature Extraction: CNN
- Classifier: CNN

SUMMARY

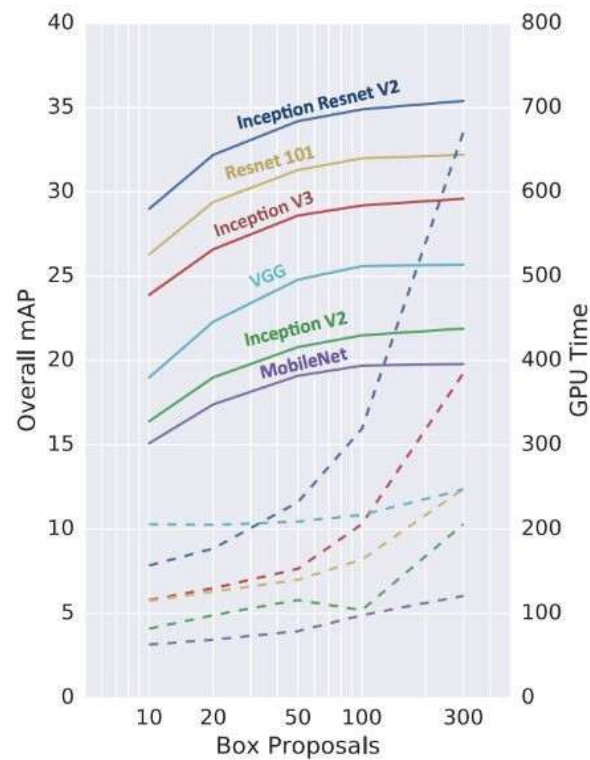
- History
 - R-CNN: Selective search → Cropped Image → CNN
 - Fast R-CNN: Selective search → Crop feature map of CNN
 - Faster R-CNN: CNN → Region-Proposal Network → Crop feature map of CNN
- Proposal Generator → Box classifier
- Best performance, but longest run-time
- End-to-end, multi-task loss
- Can use fewer proposals, but running time is dependent on proposals

<https://github.com/endernewton/tf-faster-rcnn>

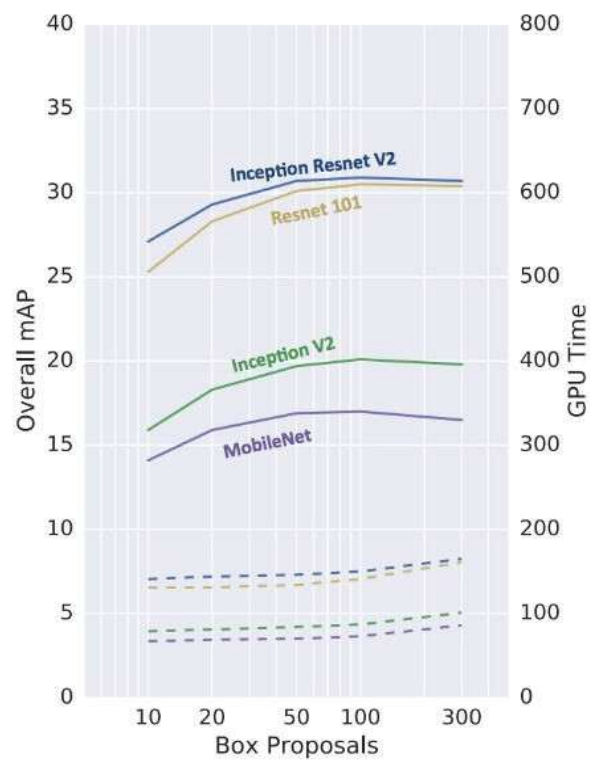


Object Detection Performance

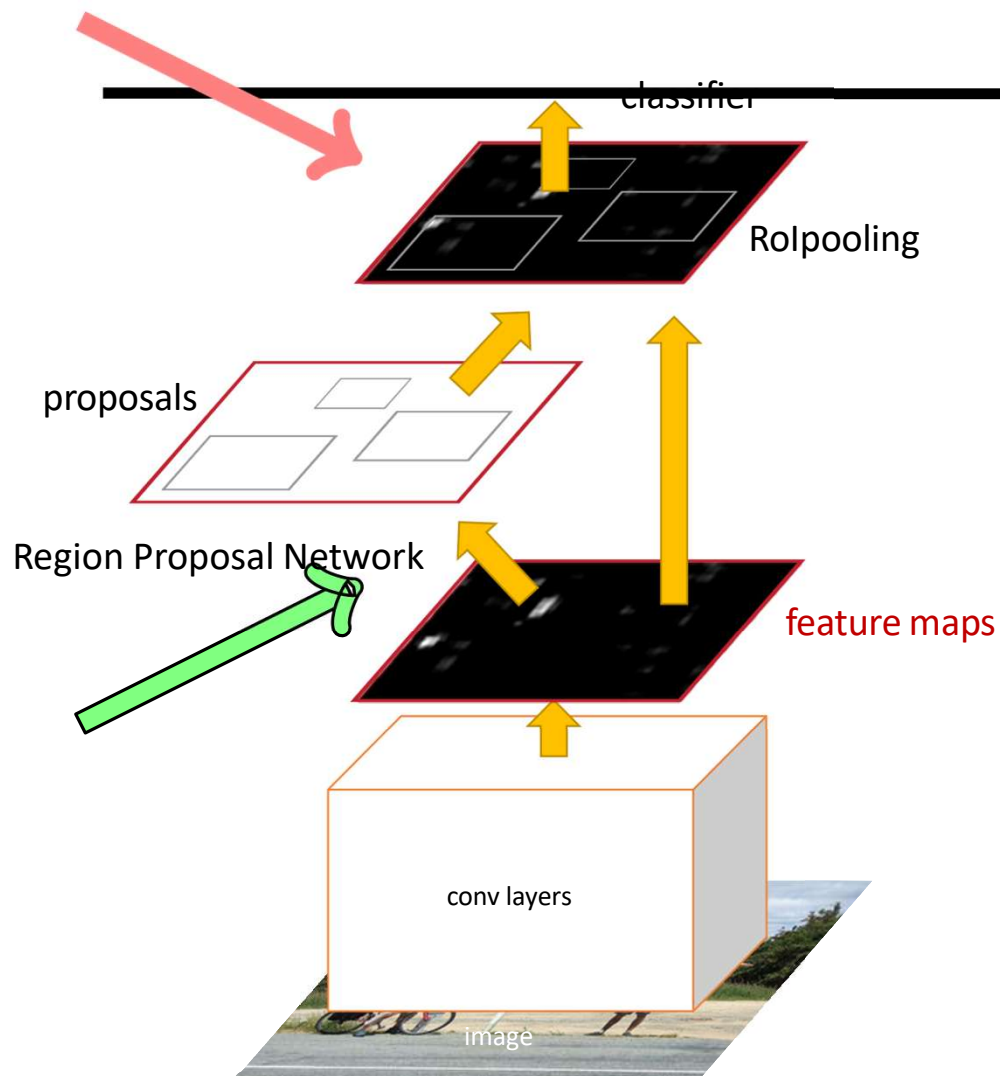
TRADEOFF: NUMBER OF PROPOSALS



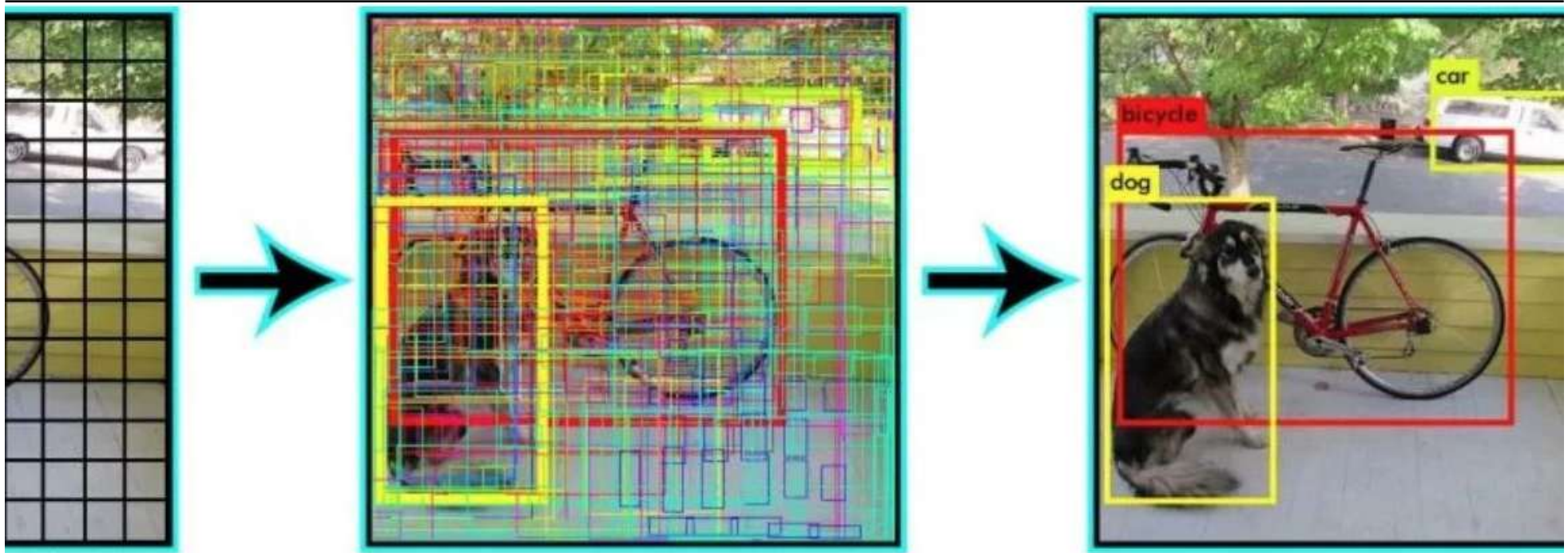
(a) FRCNN



(b) RFCN



- The approaches that we have seen so far are two stage approaches
- They involve a region proposal stage and then a classification stage
- Can we have an end-to-end architecture which does both proposal and classification simultaneously ? This is the idea behind YOLO-You Only Look Once.



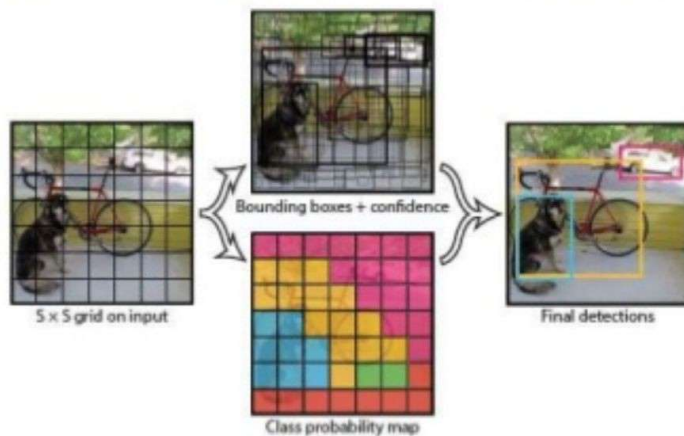
YOLO OBJECT DETECTION

Images from: <https://www.slideshare.net/TaegyunJeon1/pr12-you-only-look-once-yolo-unified-realtime-object-detection>

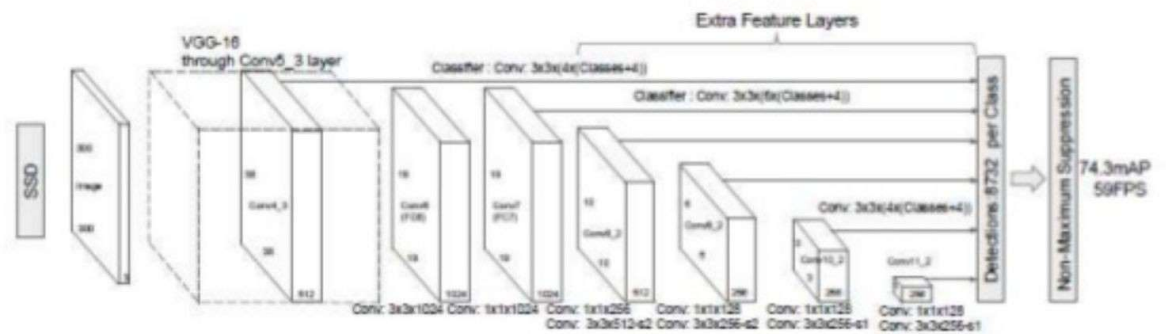
DETECTION WITHOUT PROPOSALS: YOLO/SSD

- Several techniques pose detection as a regression problem (a.k.a single shot detectors)
- Two of the most popular ones: YOLO/SSD

YOU ONLY LOOK ONCE(YOLO)



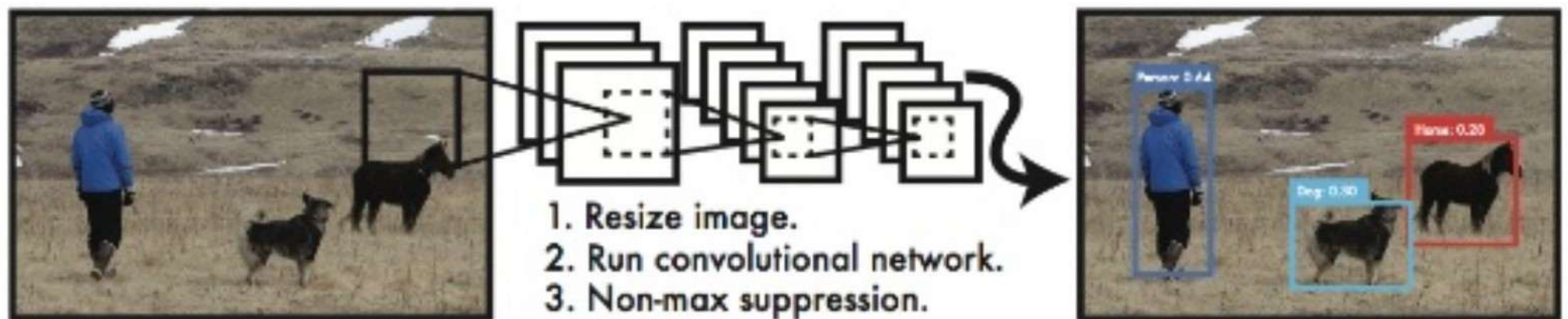
SINGLE SHOT MULTIBOX DETECTOR(SSD)



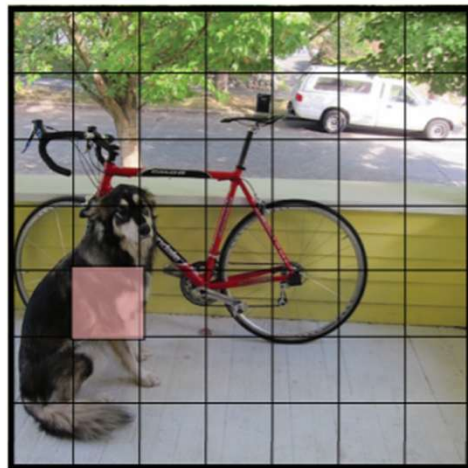
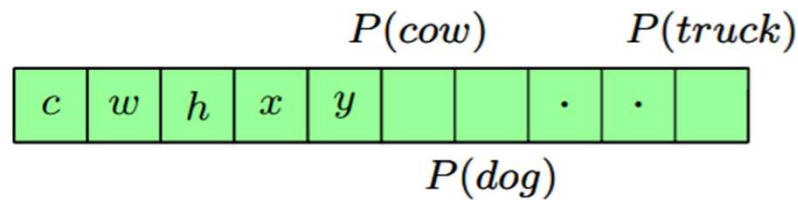
Images from: <https://www.slideshare.net/TaegyunJeon1/pr12-you-only-look-once-yolo-unified-realtime-object-detection>

YOLO

- Super fast (21~155 fps)
- Finds objects in image grids at parallel
- Only slightly worse performance than Faster R-CNN

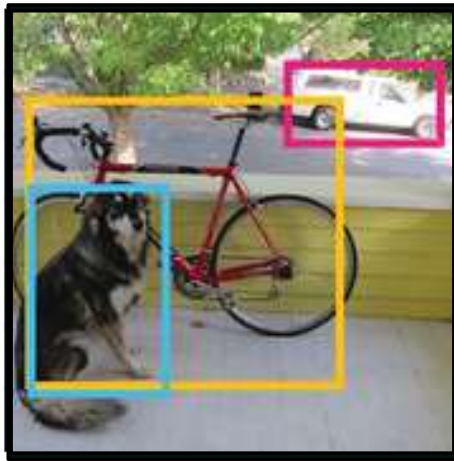


Images from: <https://www.slideshare.net/TaegyunJeon1/pr12-you-only-look-once-yolo-unified-realtime-object-detection>



$S \times S$ grid on input

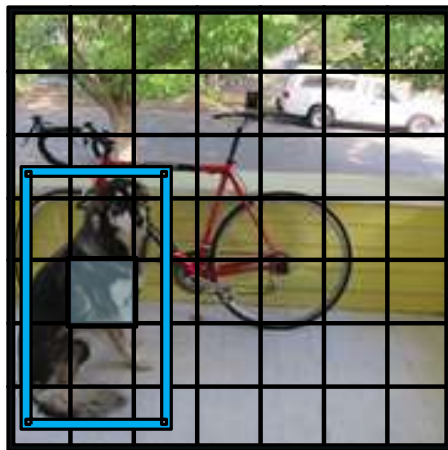
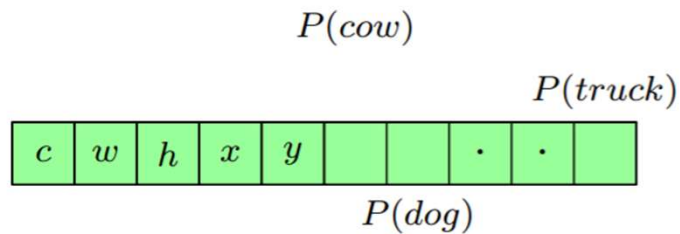
- Divide an image into $S \times S$ grids ($S=7$)
 - For each such cell we are interested in predicting 5 + k quantities
 - Probability (confidence) that this cell is indeed contained in a true bounding box.
 - Width of the bounding box
 - Height of the bounding box
 - Center (x,y) of the bounding box
 - Probability of the object in the bounding box belonging to the kth class (k - values)
 - The output layer thus contains $S \times S \times (5 + k)$ elements
-



$S \times S$ grid on input
Input Image

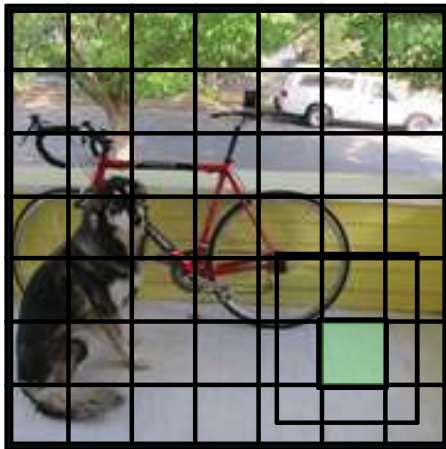
Bounding Boxes & Confidence

- How do we interpret this $(S \times S) \times 5 + k$ dimensional output?
 - For each cell, we compute a bounding box, its confidence and the object in it
 - We then retain the most confident
 - bounding boxes and the corresponding object label
-



$S \times S$ grid on input

- How do we train this network?
- Consider a cell such that the center of the true bounding box lies in it
- The network is randomly initialized and predicts some values for c, w, h, x, y, l
- We can then compute the following losses:
 - $(x - \hat{x})^2$
 - $(y - \hat{y})^2$
 - $(\sqrt{w} - \sqrt{\hat{w}})^2$
 - $(\sqrt{h} - \sqrt{\hat{h}})^2$
 - $(1 - \hat{c})^2$
 - $\sum_{i=1}^k (l_i - \hat{l}_i)^2$



$S \times S$ grid on input

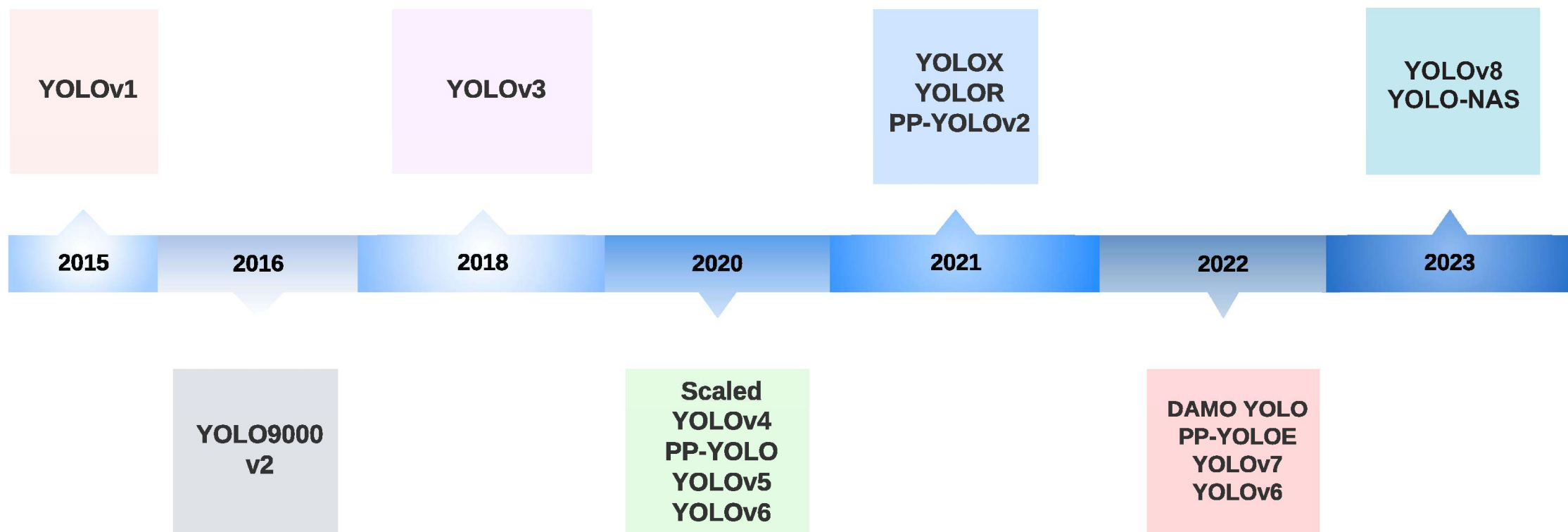
- Consider a grid which does not contain any object
 - For this grid we do not care about the predictions w, h, x, y, l
 - But we want the confidence to be low
 - So we minimize only the following loss:
$$(0 - \hat{c})^2$$
-

LIMITATIONS OF YOLO

- Groups of small objects
 - Unusual aspect ratios - struggles to generalize
 - Coarse Features (Due to multiple pooling layers from input images)
 - Localization error of bounding boxes - treats error the same for small vs large boxes
-

Method	Pascal 2007 mAP	Speed
DPM v5 RCNN	33.7 66.0 70.0 73.2	0.07 FPS — 14 sec/ image 0.05
Fast RCNN	69.0	FPS — 20 sec/ image 0.5 FPS
Faster RCNN		— 2 sec/ image 7 FPS — 140
YOLO		msec/ image 45 FPS — 22
		msec/ image

YOLO ARCHITECTURE EVOLUTION



YOLO VS YOLO V2

- YOLO: Uses InceptionNet architecture
- YOLOv2: Custom architecture - Darknet

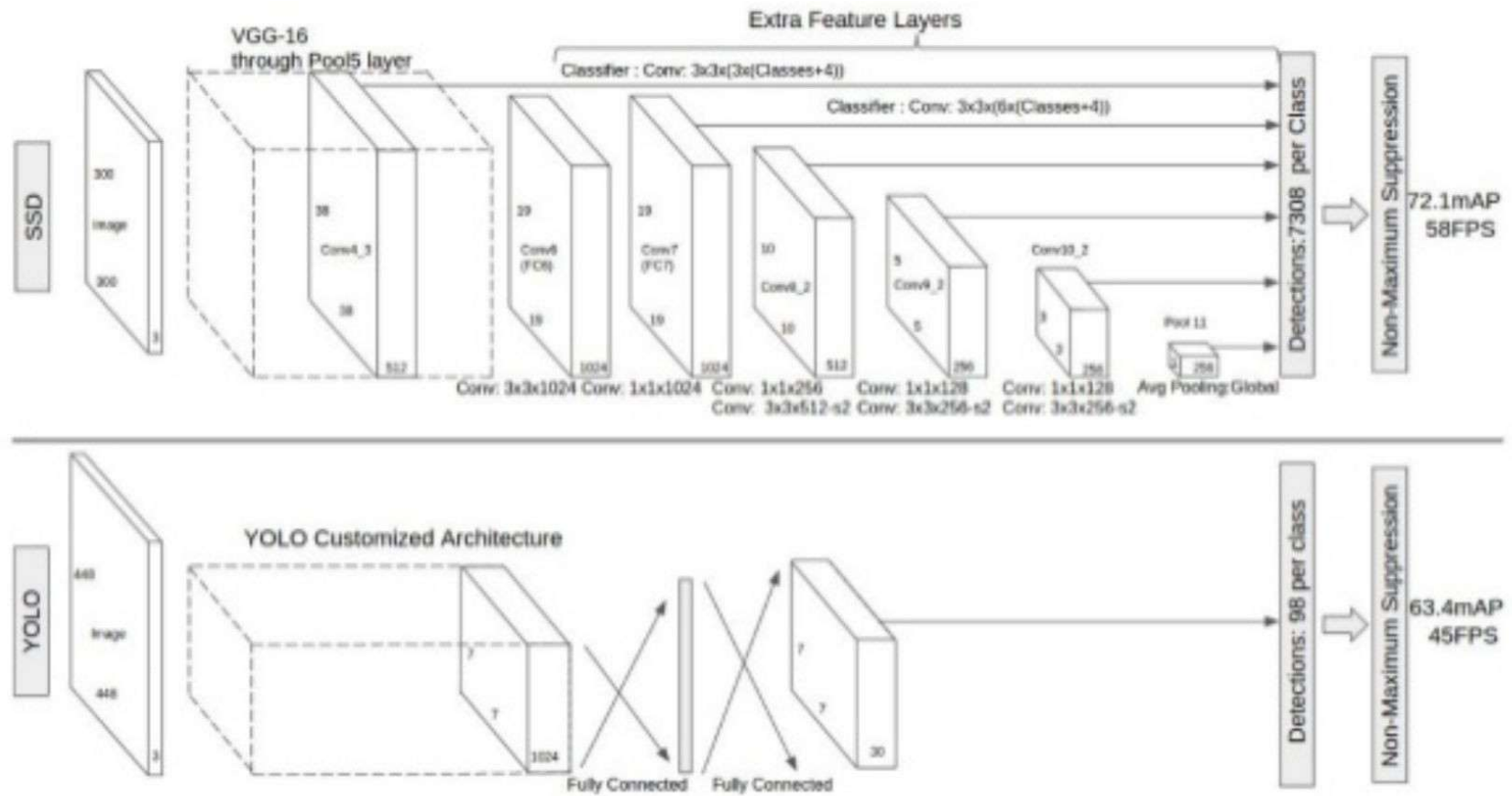
	YOLO									YOLOv2
batch norm?		✓	✓	✓	✓	✓	✓	✓	✓	✓
hi-res classifier?			✓	✓	✓	✓	✓	✓	✓	✓
convolutional?				✓	✓	✓	✓	✓	✓	✓
anchor boxes?				✓	✓					
new network?					✓	✓	✓	✓	✓	✓
dimension priors?						✓	✓	✓	✓	✓
location prediction?						✓	✓	✓	✓	✓
passthrough?							✓	✓	✓	✓
multi-scale?								✓	✓	✓
hi-res detector?										✓
VOC2007 mAP	63.4	65.8	69.5	69.2	69.6	74.4	75.4	76.8		78.6

Table from YOLO9000: Better, Faster, Stronger (<https://arxiv.org/abs/1612.08242>)

SSD

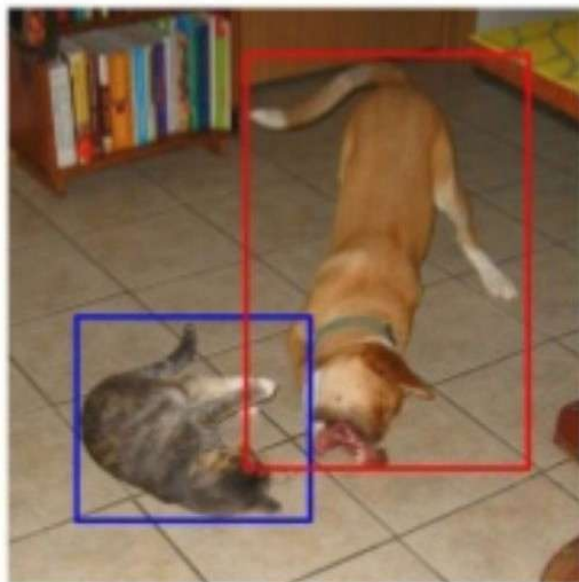
- End-to-end training (like YOLO)
 - Predicts category scores for fixed set of default bounding boxes using small convolutional filters (different from YOLO!) applied to feature maps
 - Predictions from different feature maps of different scales (different from YOLO!), separate predictors for different aspect ratio (different from YOLO!)
-

SSD VS YOLO

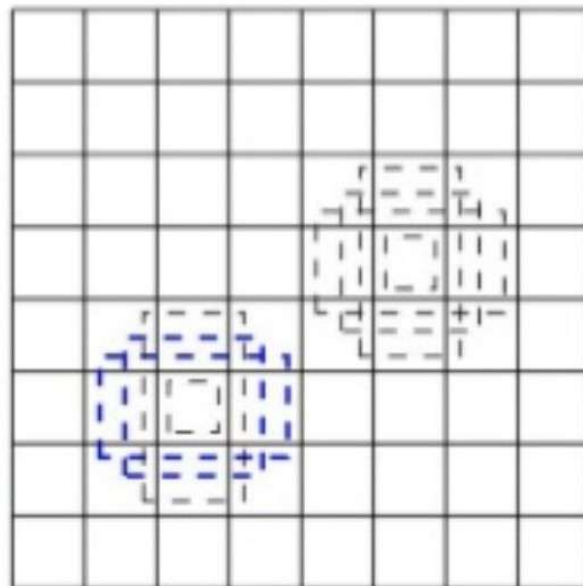


Images from: <https://www.slideshare.net/xavigiro/ssd-single-shot-multibox-detector>

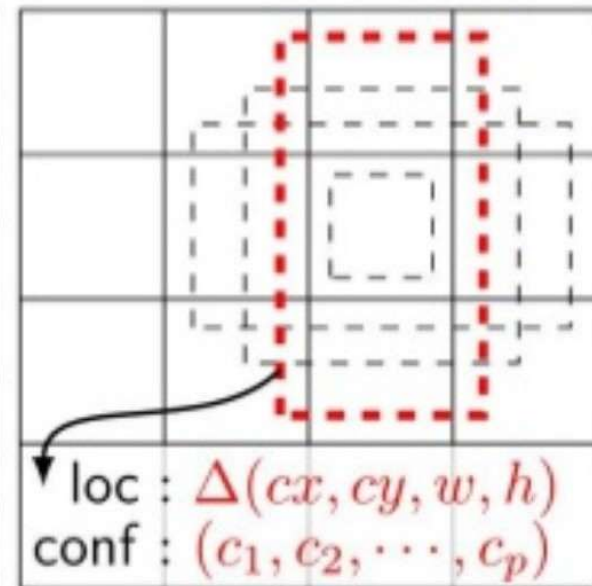
SSD



(a) Image with GT boxes



(b) 8×8 feature map



(c) 4×4 feature map

SSD

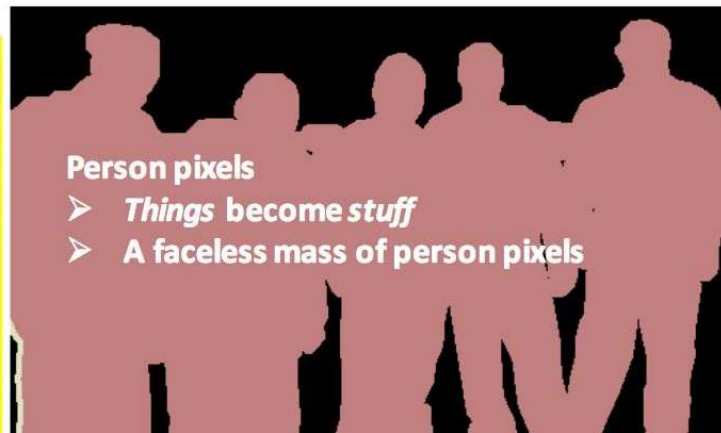
- Feature Mapping:
 - YOLO: Divides the input image into an $(S \times S)$ grid. Each grid cell is responsible for predicting bounding boxes and class probabilities for objects whose center falls within that cell.
 - SSD: Predicts objects across multiple feature maps at different resolutions (scales). This "multi-scale" approach allows it to detect objects of various sizes more naturally within the same network.
 - Network Design:
 - YOLO: Originally used a unified network with fully connected layers for final predictions (though modern versions like YOLOv11 are fully convolutional).
 - SSD: Completely convolutional in design, using auxiliary convolutional layers to assist with multi-scale feature maps.
-

SEMANTIC/INSTANCE-LEVEL SEGMENTATION

Object detection



Semantic segmentation



Instance segmentation

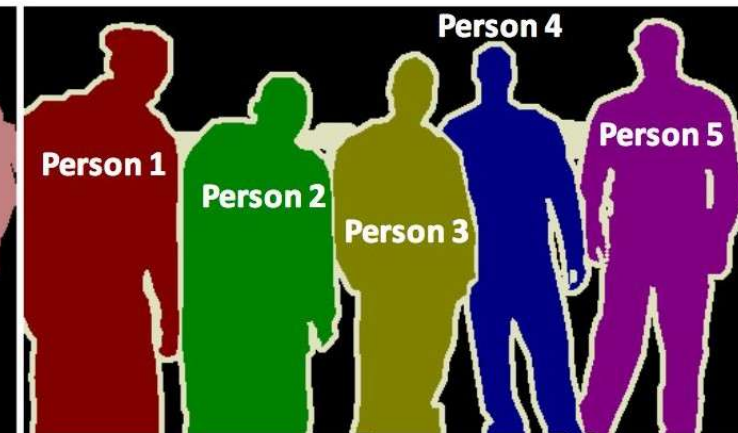


IMAGE SEGMENTATION AS DENSE PREDICTION

Unlike image classification or object detection, semantic segmentation assigns a label to every pixel in an image.

Given an image: $x \in \mathbb{R}^{H \times W \times C}$

The goal is to predict a segmentation map: $y \in 1, \dots, K^{H \times W}$

Equivalently, we estimate per-pixel posterior probabilities:

$$p(y_{i,j} = k \mid x; \theta)$$

This is a dense structured prediction problem.

CHALLENGES IN IMAGE SEGMENTATION

- High spatial resolution output
 - Strong correlation between neighboring pixels
 - Need for both:
 - **Local detail** (edges, boundaries)
 - **Global context** (object identity)
 - Classical approaches relied on:
 - Handcrafted features
 - Conditional Random Fields (CRFs)
 - U-Net provides an **end-to-end deep learning solution**.
-

LOSS FUNCTIONS FOR SEGMENTATION - PIXEL-WISE CROSS-ENTROPY

$$L_{CE} = - \sum_{i,j} \sum_{k=1}^K y_{i,j,k} \log p(y_{i,j} = k)$$

Assumes conditional independence of pixels given features.

LOSS FUNCTIONS FOR SEGMENTATION – DICE LOSS

1. Dice Loss (Overlap-Based) Dice coefficient:

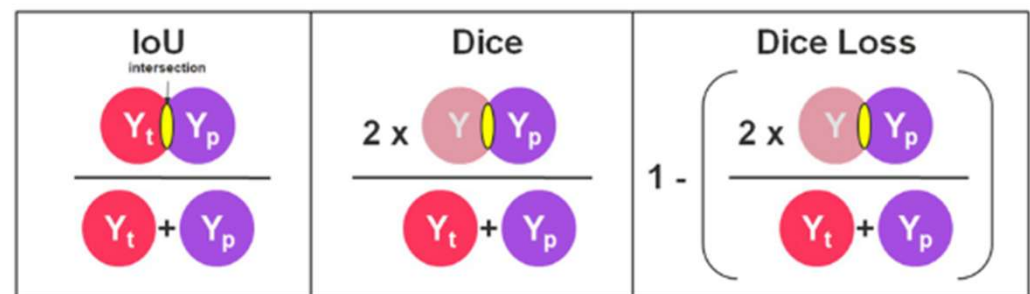
$$Dice = \frac{2 | P \cap G |}{| P | + | G |}$$

2. Soft Dice loss:

$$L_{Dice} = 1 - \frac{2 \sum pg + \epsilon}{\sum p + \sum g + \epsilon}$$

Advantages:

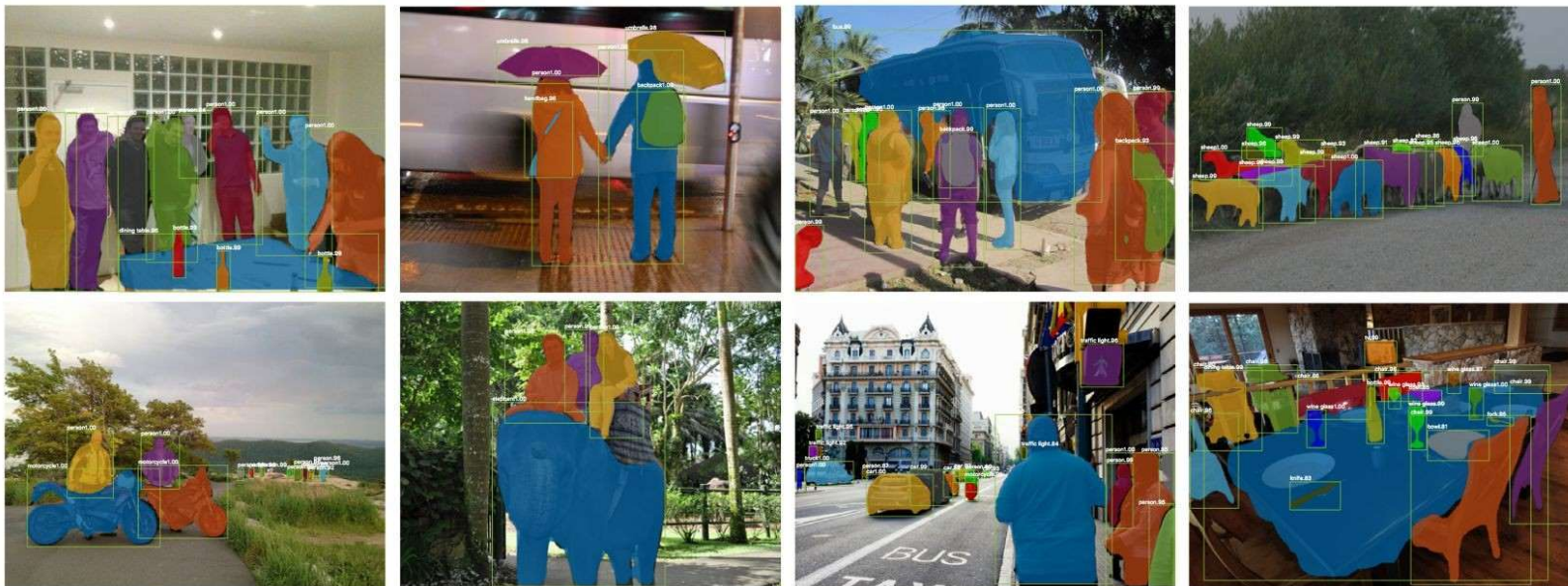
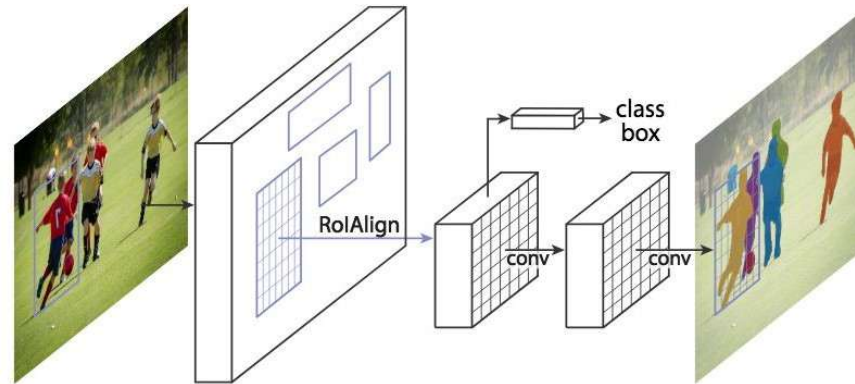
- Robust to class imbalance
- Directly optimizes region overlap



LOSS FUNCTIONS FOR SEGMENTATION – COMBINED LOSS

$$L = L_{CE} + \lambda L_{Dice}$$

MASK R-CNN



He et. al 2017

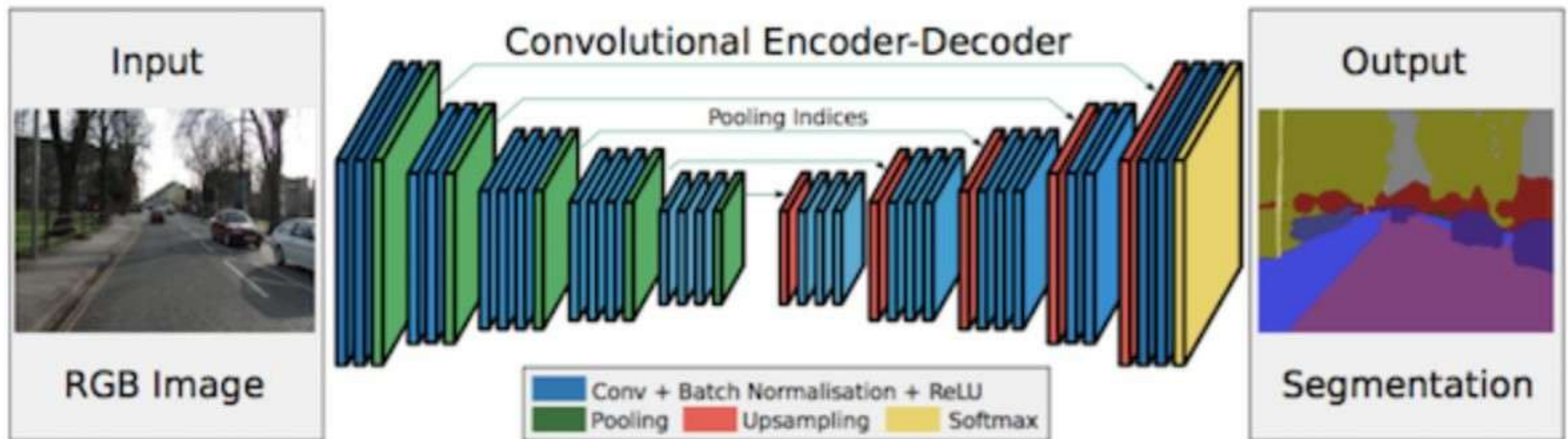
MASK R-CNN

- Backbone Architecture
- Scale Invariance (e.g. Feature Pyramid Network (FPN))
- Region Proposal Network (RPN)
- Region of interest feature alignment (RoIAlign)
- Multi-task network head
 - Box classifier
 - Box regressor
 - Mask predictor
 - Keypoint predictor



modular!

SEGNET ARCHITECTURE



SEG-NET

- Encoder-Decoder framework
 - Use dilated convolutions, a convolutional layer for dense predictions.
 - Propose 'context module' which uses dilated convolutions for multi scale aggregation.
 - Uses a novel technique to upsample encoder output which involves storing the max-pooling indices used in pooling layer. This gives reasonably good performance and is space efficient (versus FCN)
-

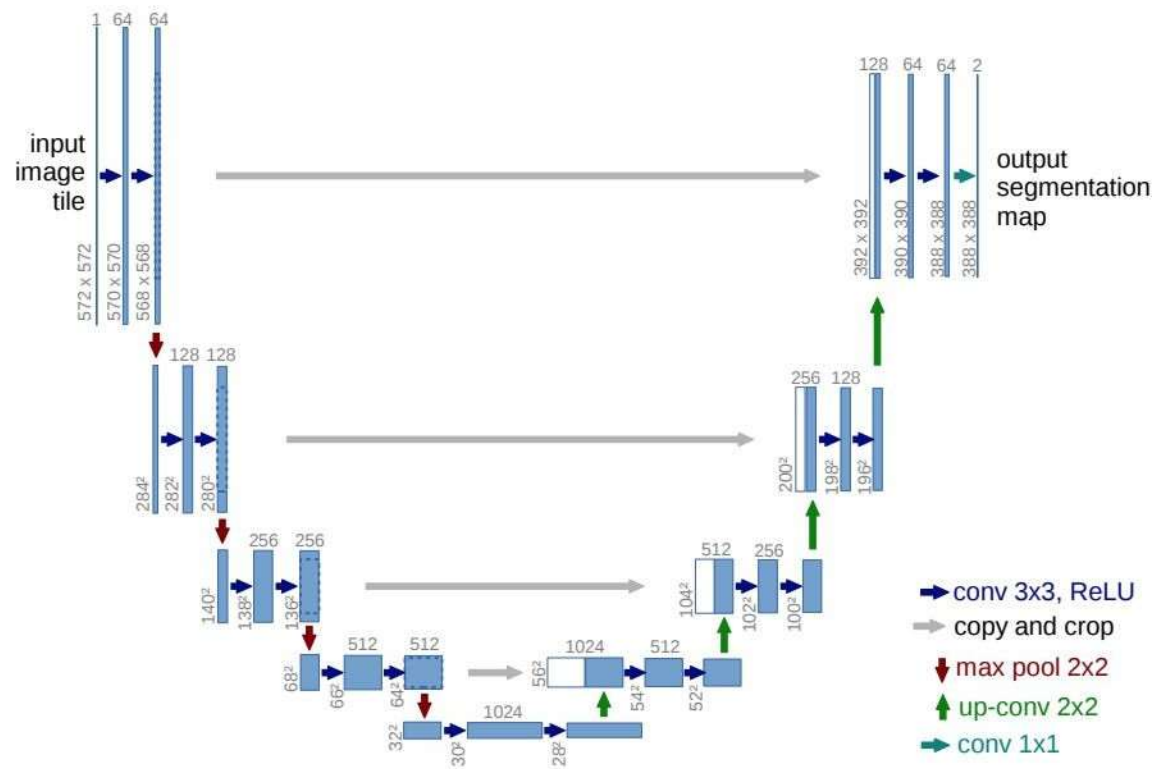
SEGET LIMITATIONS

- Applications include autonomous driving, scene understanding, etc.
 - Direct adoption of classification networks for pixel wise segmentation yields poor results mainly because max-pooling and subsampling reduce feature map resolution and hence output resolution is reduced.
 - Even if extrapolated to original resolution, lossy image is generated.
-

SEGNET VS MASK R-CNN

- Segnet
 - Dilated convolutions are very expensive, even on modern GPUs.
 - Mask R-CNN
 - Without tricks, Mask R-CNN outperforms all existing, single-model entries on every task, including the COCO 2016 challenge winners.
 - Better for pose detection
-

U-NET



Ronneberger (2015). (<https://arxiv.org/abs/1505.04597>)

U-NET: ARCHITECTURAL OVERVIEW

U-Net is a **fully convolutional encoder–decoder architecture** with **skip connections**.

Structure:

- **Encoder (contracting path)**: captures semantic context
- **Decoder (expanding path)**: recovers spatial resolution
- **Skip connections**: preserve fine-grained localization

The architecture forms a symmetric "U" shape.

ENCODER PATH (DOWNSAMPLING)

Each encoder stage consists of:

1. Two consecutive convolutions:

$$h^{(l)} = \sigma \left(W_2^{(l)} * \sigma \left(W_1^{(l)} * h^{(l-1)} + b_1^{(l)} \right) + b_2^{(l)} \right)$$

2. Spatial downsampling:

$$h_{down}^{(l)} = \text{MaxPool}(h^{(l)}, 2 \times 2) \quad h_{down}^{(l)} = \text{MaxPool}(h^{(l)}, 2 \times 2)$$

Properties:

- Channel depth doubles after each downsampling
- Spatial resolution halves

Interpretation:

- Increasing receptive field
 - Extraction of higher-level semantic features
-

DECODER PATH (UPSAMPLING)

- Each decoder stage consists of:

1. Upsampling (transposed convolution or interpolation):

$$h_{up}^{(l)} = W_u p^{(l)} \star h^{(l+1)}$$

2. Concatenation with encoder features (skip onnection):

$$z^{(l)} = \text{Concat} \left(h_{up}^{(l)}, h_{enc}^{(l)} \right)$$

Two convolutions with nonlinearities

Purpose:

- Recover spatial resolution
 - Fuse semantic and localization information
-

SKIP CONNECTIONS: MATHEMATICAL AND INTUITIVE ROLE

Skip connections pass encoder feature maps directly to the decoder:

$$z^{(l)} = \left[h_{enc}^{(l)}, h_{dec}^{(l)} \right]$$

Benefits:

- Preserve high-frequency spatial details
- Improve gradient flow
- Reduce information bottleneck

Interpretation:

- Similar to multiscale feature fusion
 - Acts as a learned alternative to CRF post-processing
-

OUTPUT LAYER AND PIXEL-WISE CLASSIFICATION

Final layer uses a 1×1 convolution:

$$s(i, j) = W_{out}h(i, j) + b_{out}$$

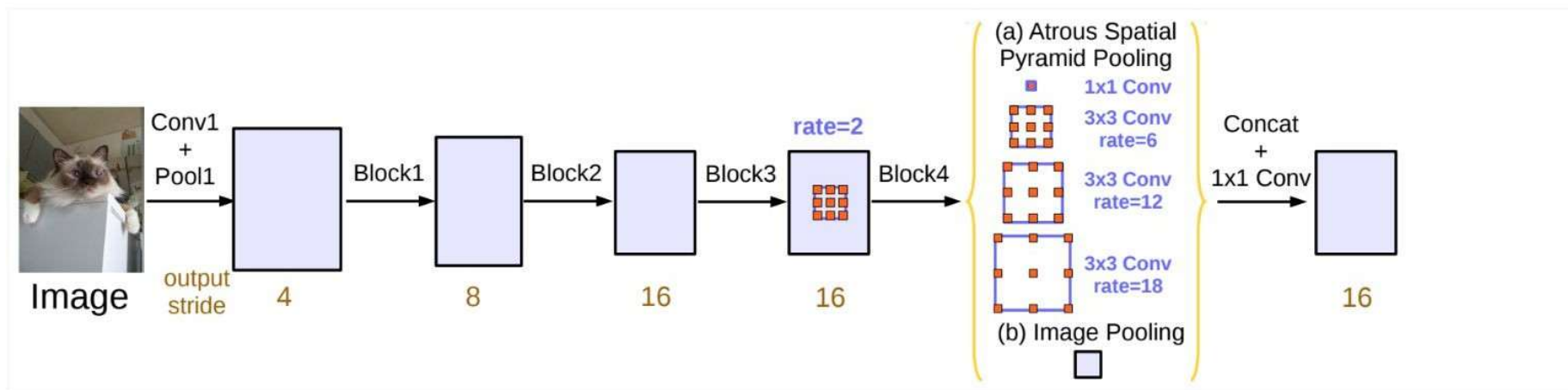
Softmax over classes:

$$p(y_{i,j} = k \mid x) = \frac{e^{s_k(i,j)}}{\sum_{k'} e^{s_{k'}(i,j)}}$$

Each pixel is classified independently conditioned on shared features.

DEEPLAB

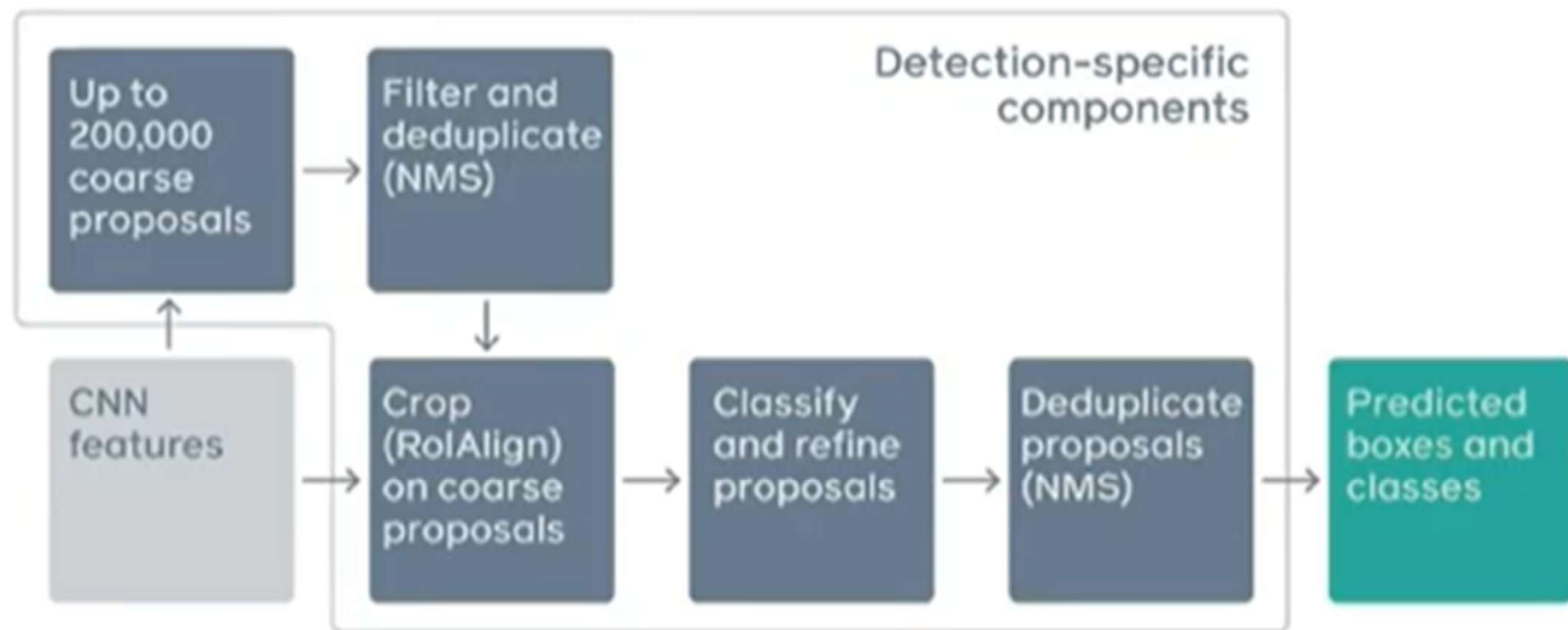
ResNet block uses atrous convolutions, uses different dilation rates to capture multi-scale context. On top of this new block, it uses Atrous Spatial Pyramid Pooling (ASPP). ASPP uses dilated convolutions with different rates as an attempt of classifying regions of an arbitrary scale.



Images from https://sthalles.github.io/deep_segmentation_network/

VISION TRANSFORMERS

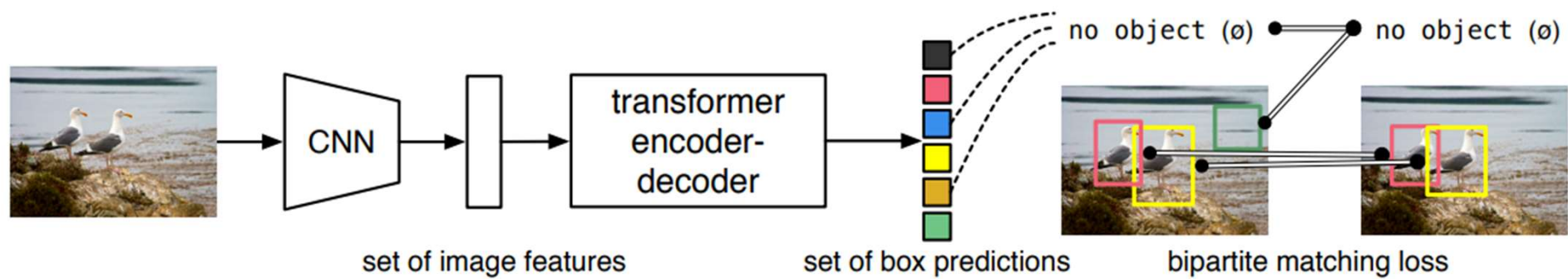
Faster R-CNN



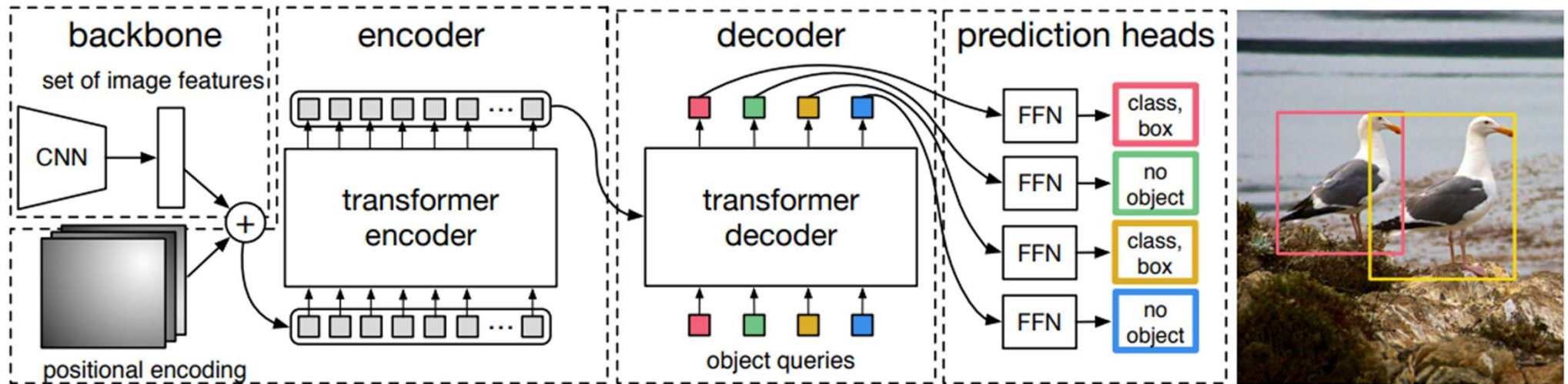
DETR



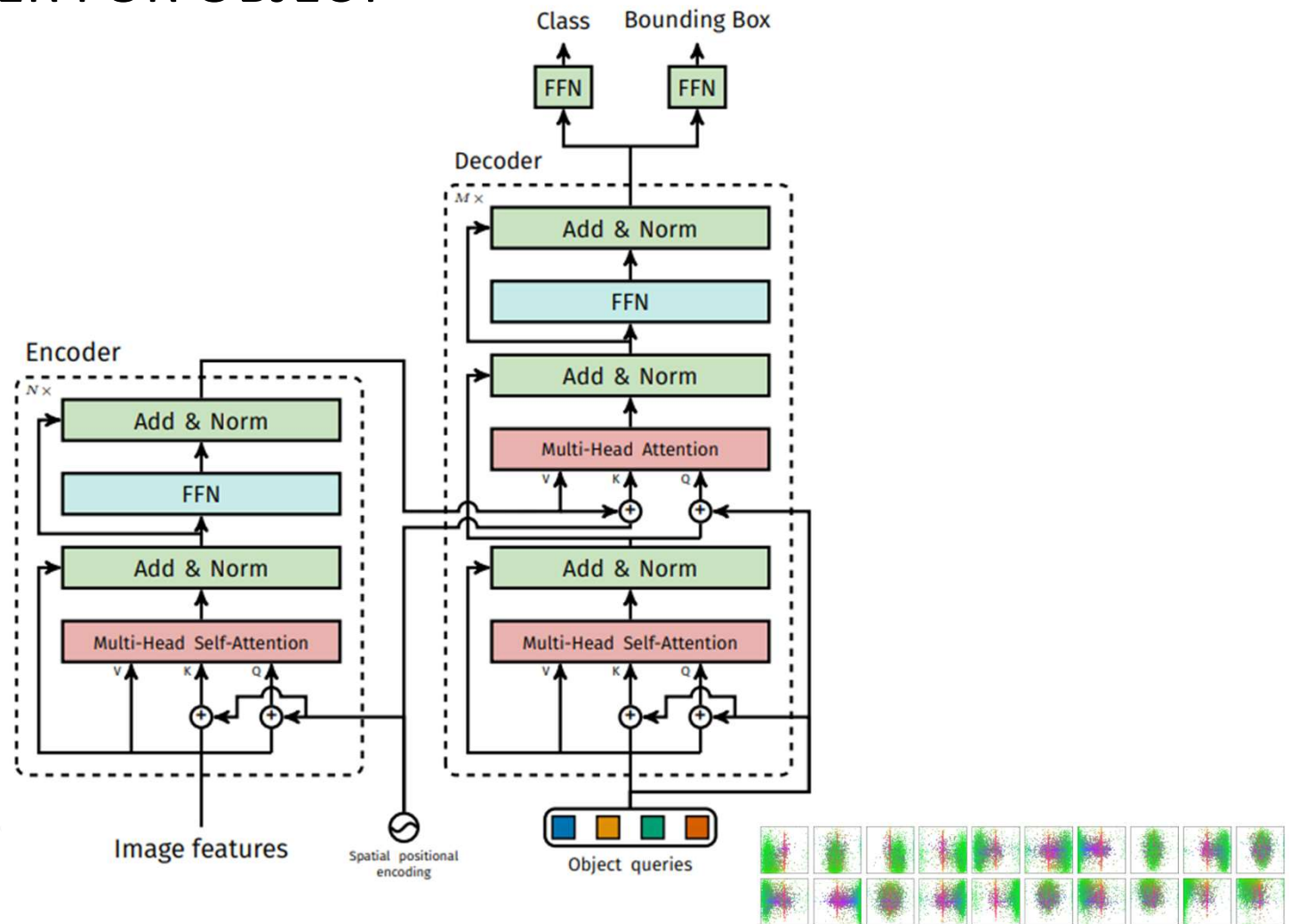
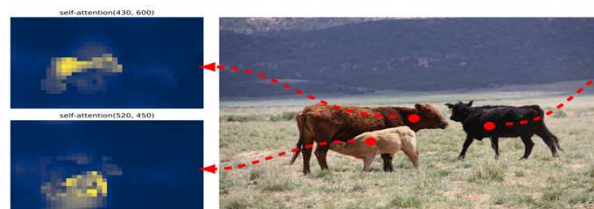
DETR



THE ENTIRE LAYOUT

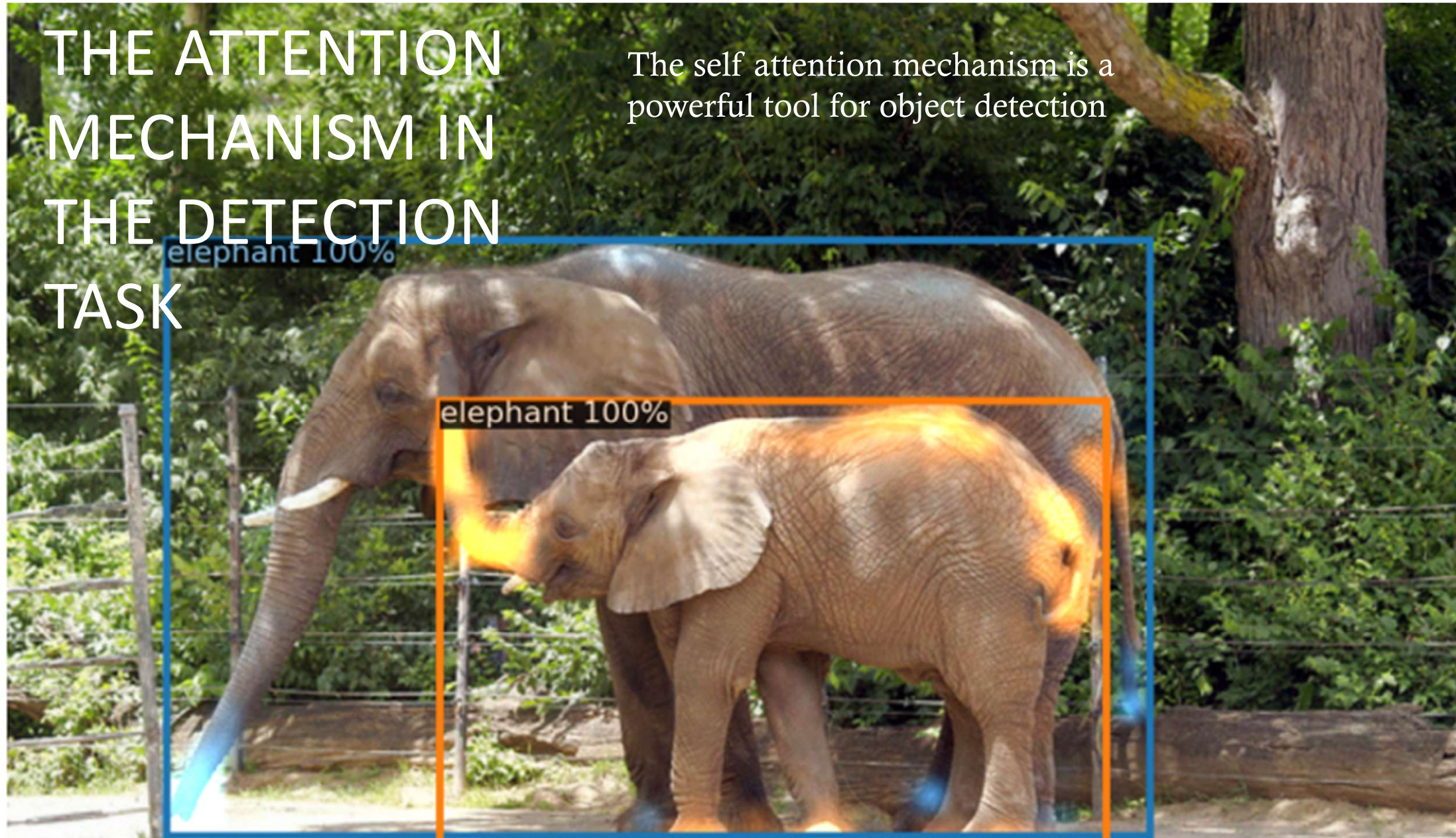


THE TRANSFORMER FOR OBJECT DETECTION



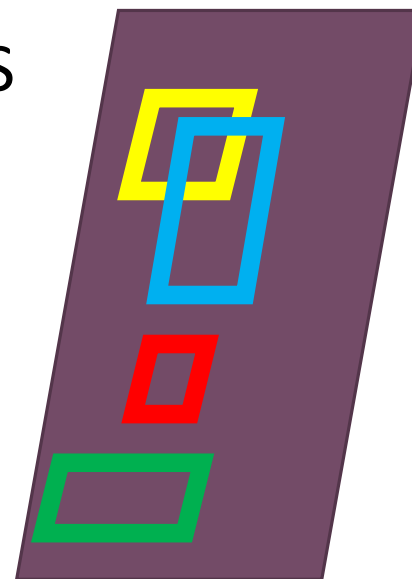
THE ATTENTION MECHANISM IN THE DETECTION TASK

The self attention mechanism is a powerful tool for object detection

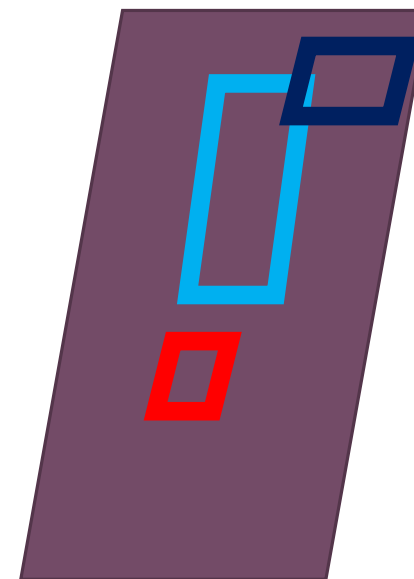


THE BI-PARTITE MATCHING LOSS

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{S}_N} \sum_i^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$



Algorithm output



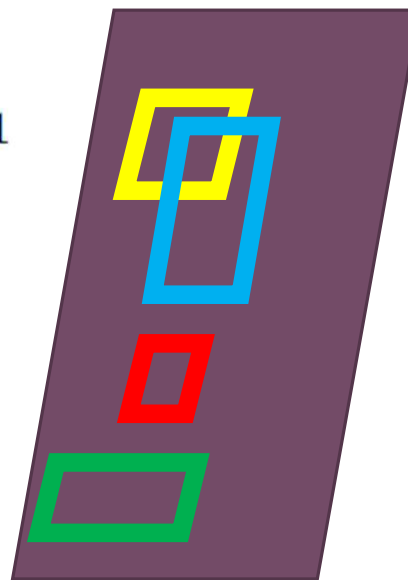
Ground truth

$$\mathcal{L}_{\text{Hungarian}}(y, \hat{y}) = \sum_{i=1}^N \left[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \right]$$

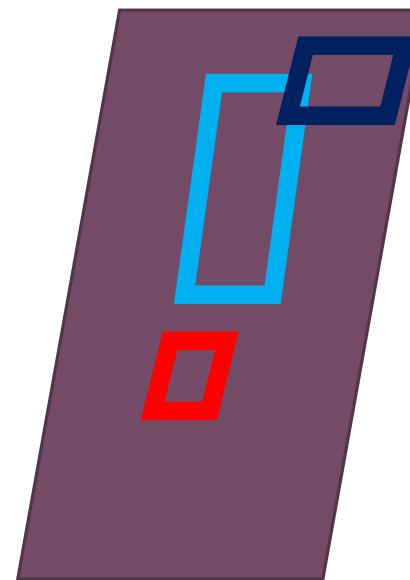
THE BI-PARTITE MATCHING LOSS

- The cost is order agnostics, enabling natural training with respect to the labeled ground truth.
- Minimum cost matching by the hungarian algorithm

$$\lambda_{\text{iou}} \mathcal{L}_{\text{iou}}(b_i, \hat{b}_{\sigma(i)}) + \lambda_{\text{L1}} \|b_i - \hat{b}_{\sigma(i)}\|_1$$

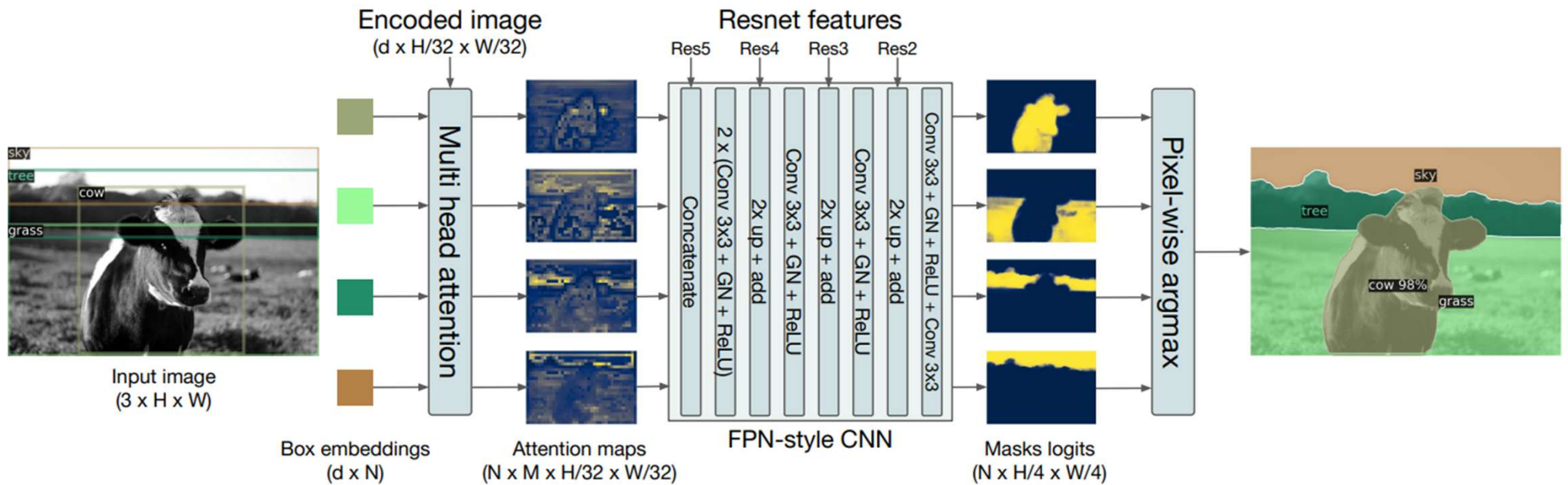


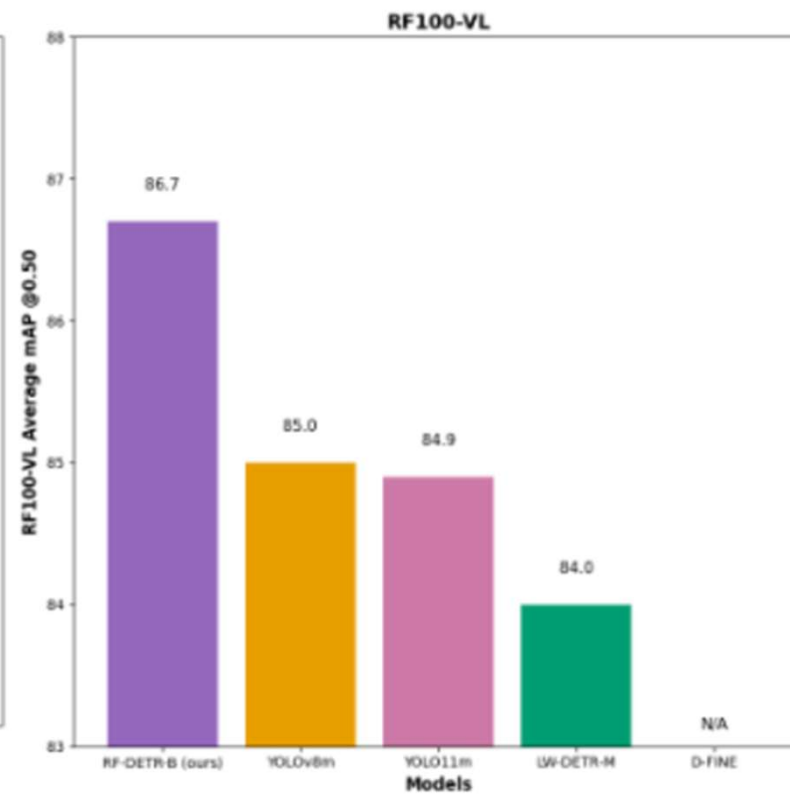
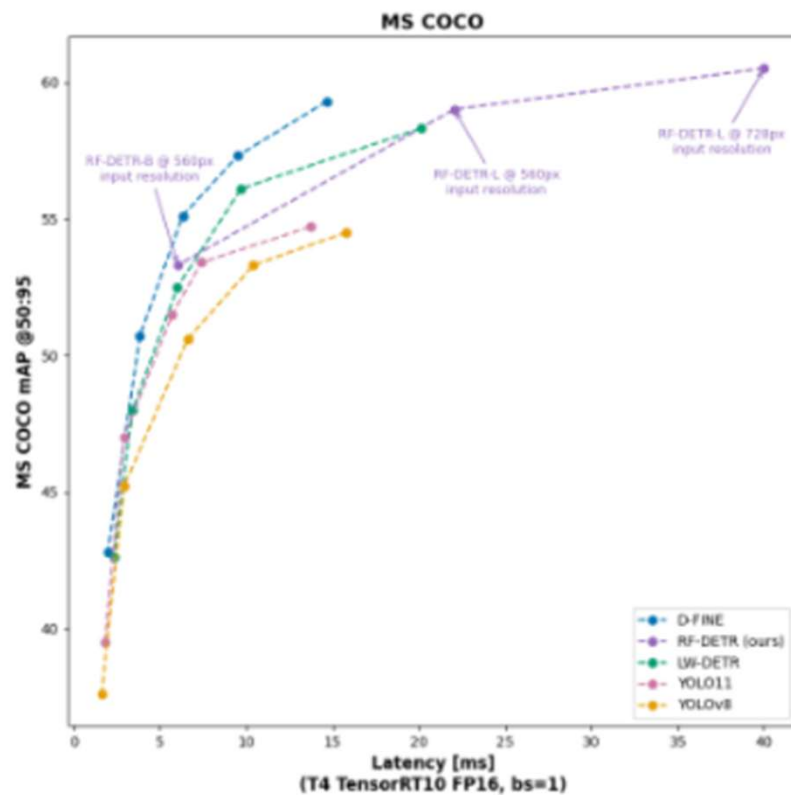
Algorithm output



Ground truth

PIXELWISE CLASSIFICATION (SEMANTIC SEGMENTATION)





RF-DETR is the first realtime model to achieve 60+ mAP on COCO and performance is comparable with other transformer-based models for smaller sizes.